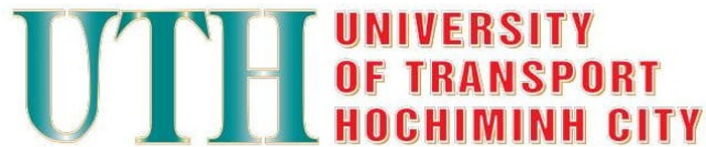


TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG HỎI ĐÁP PHÁP LUẬT TIẾNG VIỆT KẾT HỢP
RETRIEVAL-AUGMENTED GENERATION, ĐỒ THỊ TRI THỨC VÀ
CRITIC AGENT**

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

NGƯỜI HƯỚNG DẪN

ThS. Huỳnh Thanh Việt

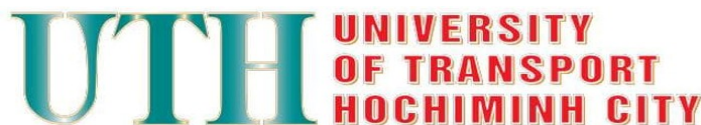
SINH VIÊN THỰC HIỆN:

Phan Quyết Tâm Phú; MSSV: 083205005445

Nguyễn Gia Huy; MSSV: 084205009123

TP. HỒ CHÍ MINH – Năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



PHAN QUYẾT TÂM PHÚ
NGUYỄN GIA HUY

**XÂY DỰNG HỆ THỐNG HỎI ĐÁP PHÁP LUẬT TIẾNG VIỆT KẾT HỢP
RETRIEVAL-AUGMENTED GENERATION, ĐỒ THỊ TRI THỨC VÀ
CRITIC AGENT**

NGÀNH: CÔNG NGHỆ THÔNG TIN
CHUYÊN NGÀNH: CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGƯỜI HƯỚNG DẪN
Th.S Huỳnh Thanh Việt
Viện Công nghệ thông tin và Điện, điện tử

TP. HỒ CHÍ MINH – Năm 2026

LỜI CAM ĐOAN

Chúng tôi tên là: Phan Quyết Tâm Phú - MSSV: 083205005445; Nguyễn Gia Huy - MSSV: 084205009123.

Ngành: Công nghệ thông tin.

Chúng tôi xin cam đoan khóa luận tốt nghiệp với đề tài "Xây dựng hệ thống hỏi đáp pháp luật tiếng Việt kết hợp Retrieval-Augmented Generation, đồ thị tri thức và Critic Agent" là kết quả nghiên cứu, thiết kế và tổng hợp của nhóm dưới sự hướng dẫn của ThS. Huỳnh Thanh Việt.

Các số liệu thực nghiệm trong khóa luận được lấy từ các artefact có trong thư mục dự án và được đối chiếu giữa bảng metric, mã nguồn và bản thảo STAIS Final. Những thông tin chưa đủ bằng chứng được trình bày dưới dạng giới hạn nghiên cứu, không được tự suy diễn thành kết quả.

Các tài liệu khoa học được trích dẫn theo chuẩn IEEE. Nhóm không sao chép trái phép kết quả của cá nhân hoặc tổ chức khác và chịu trách nhiệm về nội dung khóa luận.

TP. Hồ Chí Minh, năm 2026

Sinh viên thực hiện

Phan Quyết Tâm Phú

Nguyễn Gia Huy

TÓM TẮT

Các hệ thống hỏi đáp pháp luật dựa trên Retrieval-Augmented Generation (RAG) thường truy xuất những đoạn có độ tương đồng cao với câu hỏi rồi cung cấp chúng cho mô hình ngôn ngữ lớn. Cách tiếp cận này giúp câu trả lời dựa trên nguồn ngoài, nhưng độ liên quan ngữ nghĩa chưa bảo đảm đầy đủ cấu trúc pháp lý. Một quy định có thể phân bố qua lời dẫn, nhiều Khoản và Điểm trong cùng một Điều; chế tài chính, chế tài bổ sung và biện pháp khắc phục có thể nằm ở các phần khác nhau; một Điều cũng có thể dẫn chiếu sang Điều khác. Vì vậy, top-k child chunks có thể đúng cục bộ nhưng vẫn thiếu căn cứ cần thiết.

Khóa luận xây dựng và đánh giá một nguyên mẫu hỏi đáp pháp luật tiếng Việt kết hợp RAG, đồ thị tri thức và Critic Agent. Pipeline đã được kiểm tra từ mã nguồn hiện hành sử dụng 18 tệp DOCX để tạo 1.211 parent Articles và 10.355 child chunks; năm tệp PDF còn lại trong thư mục dữ liệu chưa đi qua chunker. Chỉ mục Qdrant dùng bộ mã hóa Base AITeamVN/Vietnamese_Embedding_v2 với vector dense 1.024 chiều, BM25 sparse và Reciprocal Rank Fusion. Đồ thị Neo4j đóng vai trò chỉ mục cấu trúc: Critic sinh câu trả lời nháp từ top-5 child, phát hiện thiếu bằng chứng trong cùng Điều hoặc qua quan hệ THAM_CHIEU, đánh giá từng ứng viên bằng cổng liên quan ngữ nghĩa, rồi chỉ sinh lại khi có bằng chứng được chấp nhận. Giao diện thực tế gồm CLI và ứng dụng Chainlit; đây là nguyên mẫu nghiên cứu, không phải hệ thống tư vấn pháp lý hoàn chỉnh.

Ba chế độ dùng chung Base retriever được so sánh trên 301 câu hỏi: Naive RAG, Article Expansion và KG-based Critic. Theo số liệu tổng hợp trong nguồn FinalFix, LCR toàn bộ lần lượt là 43,77%, 72,72% và 81,33%; số lượt gọi mô hình trung bình được Metrics-Fix hiệu chỉnh là 1,0, 1,0 và 5,7. Các chỉ số LCR, RAGAS và chi phí tổng hợp được ghi rõ là số liệu do nguồn báo cáo vì output Critic hiện chỉ có 23/301 record và chưa đủ để tái tạo toàn bộ phép chấm. Trái lại, corpus, cấu trúc chỉ mục và kiểm tra xác định của testset đã được đối chiếu trực tiếp với mã nguồn hoặc artefact. Kết quả mô tả cho thấy khôi phục toàn Điều cải thiện mạnh độ bao phủ so với top-k child; Critic tiếp tục cải thiện trên nhóm cần dẫn chiếu, nhưng phải trả chi phí token và số lượt gọi cao hơn. Kết luận chỉ áp dụng cho snapshot và protocol đã có, không thay thế thẩm định của chuyên gia luật.

Từ khóa: hỏi đáp pháp luật; Retrieval-Augmented Generation; đồ thị tri thức; Critic Agent; truy xuất lại; Legal Completeness Rate.

MỤC LỤC

1. TỔNG QUAN.....	1
1.1. Bối cảnh và động cơ nghiên cứu.....	1
1.2. Phát biểu bài toán.....	1
1.3. Mục tiêu nghiên cứu.....	1
1.4. Câu hỏi nghiên cứu.....	1
1.5. Phạm vi nghiên cứu.....	1
1.6. Đóng góp của khóa luận.....	1
1.7. Bố cục khóa luận.....	1
2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN.....	1
2.1. Retrieval-Augmented Generation.....	1
2.2. Biểu diễn dense và cosine similarity.....	1
2.3. Truy xuất thưa bằng BM25.....	1
2.4. Reciprocal Rank Fusion.....	1
2.5. Chunking theo cấu trúc Điều - Khoản - Điểm.....	1
2.6. Đồ thị tri thức pháp luật.....	1
2.7. Critic Agent và bổ sung bằng chứng có chọn lọc.....	1
2.8. Nghiên cứu liên quan về legal QA và RAG.....	1
2.9. Chỉ tiêu đánh giá.....	1
2.10. Khoảng trống nghiên cứu.....	1
3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	1
3.1. Yêu cầu chức năng.....	1
3.2. Yêu cầu phi chức năng.....	1
3.3. Kiến trúc tổng thể.....	1
3.4. Thiết kế dữ liệu văn bản.....	1
3.5. Thiết kế chỉ mục Qdrant.....	1
3.6. Thiết kế Knowledge Graph.....	1
3.7. Thiết kế ba chế độ RAG.....	1
3.8. Thuật toán Critic Agent.....	1
3.9. Thiết kế bộ đánh giá.....	1
3.10. Thiết kế bảo mật và quản trị dữ liệu.....	1
4. CÀI ĐẶT VÀ THỰC NGHIỆM.....	1
4.1. Tổ chức mã nguồn.....	1

4.2. Môi trường và mô hình.....	1
4.3. Corpus và chỉ mục.....	1
4.4. Xây dựng và nạp đồ thị.....	1
4.5. Quy trình thao tác và các kịch bản đầu vào.....	1
4.6. Quy trình chạy đánh giá.....	1
4.7. Chấm Legal Completeness.....	1
4.8. Kiểm thử phần mềm và kiểm soát lỗi.....	1
4.9. Khả năng tái lập.....	1
5. KẾT QUẢ VÀ THẢO LUẬN.....	1
5.1. Độ tin cậy của bộ testset.....	1
5.2. Kết quả truy xuất ban đầu của Base retriever.....	1
5.3. Legal Completeness Rate.....	1
5.4. Kết quả RAGAS.....	1
5.5. Precision, recall và độ dài ngữ cảnh theo Điều.....	1
5.6. Chi phí token và số lượt gọi mô hình.....	1
5.7. Trả lời các câu hỏi nghiên cứu.....	1
5.8. Phân tích lỗi và rủi ro.....	1
5.9. Giới hạn nghiên cứu.....	1
5.10. Hàm ý thiết kế.....	1
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	1
6.1. Kết luận.....	1
6.2. Hướng phát triển.....	1
TÀI LIỆU THAM KHẢO.....	1
PHỤ LỤC A. BẢNG ĐỐI CHIẾU BẢNG CHỨNG.....	1
PHỤ LỤC B. DANH SÁCH GIỚI HẠN TÁI LẬP CẦN BỔ SUNG.....	1
PHỤ LỤC C. TUYÊN BỐ VỀ VIỆC SỬ DỤNG CÔNG CỤ AI.....	1



DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc offline và online với Base hybrid retriever dùng chung cho ba chế độ.....	1
Hình 2. Vòng đời dữ liệu từ văn bản nguồn đến chỉ mục, đồ thị, testset và trả lời trực tuyến.....	1
Hình 3. Chuẩn hóa cấu trúc và biểu diễn truy xuất dense/BM25.....	1
Hình 4. Pipeline truy xuất lai dense/BM25, RRF và ba chế độ RAG.....	1
Hình 5. Lưu đồ quyết định của Critic: phát hiện thiếu, semantic gate và sinh lại có điều kiện.....	1
Hình 6. Quy trình kiểm định hai lớp cho testset và các artefact đầu ra cần lưu.....	1
Hình 7. Bốn kiểu khoảng trống bằng chứng và hành vi của Critic trong nguyên mẫu.....	1
Hình 8. Legal Completeness Rate do FinalFix báo cáo cho ba chế độ dùng chung Base retriever.....	1
Hình 9. Đánh đổi LCR, pipeline token và số lượt gọi mô hình của cấu hình Base.....	1

DANH MỤC BẢNG

Bảng 1. Cấu trúc định danh chunk và vai trò trong pipeline.....	1
Bảng 2. Yêu cầu người dùng và tiêu chí chấp nhận.....	1
Bảng 3. Các bước xử lý một truy vấn từ đầu vào đến ngữ cảnh sinh.....	1
Bảng 4. Payload tối thiểu của child và parent chunk.....	1
Bảng 5. Các nhóm nút và quan hệ chính trong đồ thị.....	1
Bảng 6. So sánh vận hành của ba chế độ.....	1
Bảng 7. Bảng quyết định các nhánh vận hành đã xác minh và đề xuất.....	1
Bảng 8. Phân bố bộ câu hỏi đánh giá.....	1
Bảng 9. Chín kiểm tra dùng để kiểm định bộ testset.....	1
Bảng 10. Các module chính của hệ thống.....	1
Bảng 11. Cấu hình thực nghiệm chính.....	1
Bảng 12. Các kịch bản nhập câu hỏi và hành vi mong đợi.....	1
Bảng 13. Case study 1 — No-gap, khung trace cần bổ sung.....	1
Bảng 14. Case study 2 — Same-Article gap, khung trace cần bổ sung.....	1
Bảng 15. Case study 3 — Cross-Article reference, khung trace cần bổ sung.....	1
Bảng 16. Kết quả kiểm định testset do tài liệu phương pháp báo cáo.....	1
Bảng 17. Metric retrieval Base theo hai snapshot bằng chứng.....	1
Bảng 18. Legal Completeness Rate trên 301 câu hỏi.....	1
Bảng 19. Kết quả RAGAS Base do FinalFix báo cáo.....	1
Bảng 20. Chỉ số context ở cấp Điều.....	1
Bảng 21. Chỉ số tài nguyên trung bình trên mỗi câu hỏi.....	1
Bảng 22. Nguồn nội bộ dùng cho các khẳng định thực nghiệm.....	1

1. TỔNG QUAN

1.1. Bối cảnh và động cơ nghiên cứu

Văn bản pháp luật là nguồn tri thức có tính cấu trúc cao, thay đổi theo thời gian và đòi hỏi người đọc đồng thời xác định đúng văn bản, đúng Điều, đúng Khoản, đúng Điểm, phạm vi áp dụng và các dẫn chiếu liên quan. Khi người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên, từ khóa trong câu hỏi có thể khác với cách diễn đạt trong văn bản. Một hệ thống tra cứu chỉ dựa trên tìm kiếm từ khóa vì vậy có thể bỏ sót quy định dùng từ đồng nghĩa hoặc mô tả pháp lý khác. Ngược lại, một mô hình ngôn ngữ lớn có thể tạo câu trả lời trôi chảy nhưng không có cơ chế bảo đảm mọi kết luận đều dựa trên văn bản hiện hành.

RAG kết hợp bộ truy xuất với mô hình sinh, cho phép mô hình điều kiện hóa câu trả lời trên các tài liệu lấy từ kho dữ liệu ngoài [1]. Kho tri thức có thể cập nhật độc lập với tham số của mô hình, đồng thời các đoạn ngữ cảnh được truy xuất tạo cơ sở để kiểm tra câu trả lời. Đối với tiếng Việt, bài toán hỏi đáp pháp luật đã được nghiên cứu ở mức truy xuất Điều luật, trong đó câu hỏi được ánh xạ tới các Điều liên quan [2]. Các hệ thống pháp lý thực tế cũng đã kết hợp tìm kiếm thưa, biểu diễn dense và reranking [3]. Tuy nhiên, độ liên quan của từng đoạn chưa đồng nghĩa với độ đầy đủ của căn cứ pháp lý.

Trong dữ liệu pháp luật Việt Nam, một Điều thường có lời dẫn và nhiều Khoản; mỗi Khoản có thể tiếp tục chứa nhiều Điểm. Quy định về một hành vi có thể đặt mức phạt tiền ở một Khoản, hình thức xử phạt bổ sung ở Khoản khác và biện pháp khắc phục hậu quả ở phần cuối Điều. Một quy tắc còn có thể dẫn chiếu sang Điều khác bằng các cụm như “theo quy định tại Điều...”. Nếu chỉ lấy top-k chunk riêng lẻ, hệ thống có thể trả lời đúng phần mức phạt nhưng bỏ sót tịch thu phương tiện, đình chỉ hoạt động hoặc nghĩa vụ khắc phục. Nếu luôn mở rộng toàn bộ nhiều Điều, ngữ cảnh tăng nhanh và có nguy cơ chứa chi tiết không liên quan; nghiên cứu về ngữ cảnh dài cho thấy khả năng sử dụng bằng chứng của mô hình còn phụ thuộc vị trí của bằng chứng trong chuỗi đầu vào [4].

Đồ thị tri thức cung cấp một cách biểu diễn khác: thực thể pháp lý trở thành nút, còn quan hệ cấu trúc và dẫn chiếu trở thành cạnh. Các hệ thống truy xuất dựa trên đồ thị có khả năng kết nối bằng chứng nằm ở nhiều đoạn [5]. Từ quan sát đó, khóa luận đặt Critic Agent sau bước sinh nháp. Critic không thay thế retriever mà dùng cấu trúc đồ thị để trả lời câu hỏi kiểm tra: các Điều đang xét còn thiếu phần nào, có chế tài kếp hay không, có dẫn chiếu ra Điều khác hay không, và phần được phát hiện có thật sự liên quan đến câu hỏi hay không.

1.2. Phát biểu bài toán

Đầu vào của hệ thống là một câu hỏi pháp luật tiếng Việt và một kho văn bản đã được chuẩn hóa. Đầu ra mong muốn là câu trả lời có căn cứ, đúng trọng tâm và bao phủ các ý pháp lý bắt buộc. Bài toán không chỉ là tìm đoạn giống câu hỏi nhất. Hệ thống cần giải quyết đồng thời bốn yêu cầu:

1. Truy xuất đúng đoạn nhỏ để tránh đưa quá nhiều nội dung không liên quan vào prompt.
2. Khôi phục cấu trúc toàn Điều khi câu trả lời cần nhiều Khoản hoặc Điểm trong cùng Điều.
3. Lần theo dẫn chiếu để tìm Điều liên quan nằm ngoài kết quả top-k ban đầu.
4. Kiểm soát chi phí bằng cách chỉ bổ sung và sinh lại khi phần mới thực sự hữu ích.

Khóa luận xem “đầy đủ pháp lý” là khả năng câu trả lời thể hiện đúng các required facts đã gắn với từng câu hỏi đánh giá. Đây là tiêu chí vận hành cho thí nghiệm, không phải kết luận về giá trị tư vấn pháp lý của hệ thống. Bộ required facts và đáp án tham chiếu chưa được chuyên gia luật xác nhận độc lập, vì vậy được xem là silver standard.

1.3. Mục tiêu nghiên cứu

Mục tiêu tổng quát là xây dựng và đánh giá kiến trúc RAG nhận biết cấu trúc cho hỏi đáp pháp luật tiếng Việt. Các mục tiêu cụ thể gồm:

- Xây dựng bộ tách văn bản theo Điều - Khoản - Điểm và cơ chế parent/child chunk.
- Tạo chỉ mục truy xuất lai kết hợp embedding tiếng Việt và BM25.
- Trích xuất đồ thị tri thức pháp luật, đặc biệt là quan hệ cấu trúc, chế tài và THAM_CHIEU.
- Thiết kế Critic Agent phát hiện bằng chứng thiếu, lọc ngữ nghĩa và sinh lại có điều kiện.
- So sánh công bằng ba chế độ xử lý trên cùng truy vấn, corpus và Base retriever dùng chung.
- Đánh giá cả chất lượng câu trả lời, chất lượng truy xuất và chi phí tài nguyên.

1.4. Câu hỏi nghiên cứu

Khóa luận trả lời ba câu hỏi nghiên cứu:

- **RQ1:** Naive RAG, mở rộng toàn Điều và KG-based critic khác nhau như thế nào về Legal Completeness Rate, đặc biệt trên câu hỏi đa cấu trúc và tham chiếu chéo?
- **RQ2:** Việc khôi phục toàn Điều và bổ sung bằng chứng theo cấu trúc ảnh hưởng như thế nào đến Article precision, Article recall, RAGAS và độ dài context trên Base retriever?
- **RQ3:** Mức cải thiện LCR của Critic Agent đi kèm sự đánh đổi nào về pipeline token, final-call token và số lượt gọi mô hình?

1.5. Phạm vi nghiên cứu

Corpus trong workspace gồm 23 văn bản pháp luật Việt Nam, trong đó 18 file DOCX đi qua pipeline chunking và năm file PDF được lưu làm nguồn nhưng chưa được parse vào chỉ mục thí nghiệm. Nội dung tập trung vào sở hữu trí tuệ, công nghệ thông tin, an ninh mạng, dữ liệu cá nhân, giao dịch điện tử, viễn thông, thương mại điện tử và xử phạt hành chính có liên quan. Kết quả không đại diện cho toàn bộ hệ thống pháp luật Việt Nam.

Khóa luận tập trung vào pipeline dữ liệu, Base hybrid retrieval, đồ thị, Critic Agent, đánh giá offline và nguyên mẫu CLI/Chainlit. Tư vấn pháp lý chính thức, vận hành production, bảo đảm SLA và kiểm định bởi luật sư không thuộc phạm vi kết luận. Các thử nghiệm fine-tuned và GTE chỉ được nhắc như lịch sử nghiên cứu: workspace không có checkpoint fine-tuned, còn collection mang tên GTE có kích thước 1.024 không khớp cấu hình chính thức 768 chiều, nên cả hai không đủ điều kiện làm trực kết quả có thể tái lập.

1.6. Đóng góp của khóa luận

Đóng góp thứ nhất là quy trình chunking bảo toàn cấu trúc pháp luật. Mỗi phần nhỏ nhất có thể truy xuất được gắn định danh vị trí pháp lý, đồng thời các child chunk cùng Điều được liên kết với parent chunk toàn Điều. Cách tổ chức này tách hai mục tiêu thường xung đột: độ chính xác của truy xuất và độ đầy đủ của ngữ cảnh.

Đóng góp thứ hai là đồ thị tri thức được xây dựng bằng chiến lược trích xuất hai lượt và hậu xử lý rule-based. Đồ thị không chỉ lưu Điều, Khoản, Điểm mà còn biểu diễn hành vi, chế tài và cạnh THAM_CHIEU. Placeholder node giúp ghi nhận một đích dẫn chiếu chưa có trong đồ thị thay vì làm mất quan hệ.

Đóng góp thứ ba là Critic Agent bổ sung bằng chứng theo nhu cầu. Thay vì luôn mở rộng mọi Điều, Critic phát hiện ba nhóm thiếu sót: chế tài kẹp trong cùng Điều, thiếu thành phần cấu trúc của Điều, và dẫn chiếu chéo giữa các Điều. Ứng viên còn phải qua cổng liên quan ngữ nghĩa trước khi đi vào ngữ cảnh sinh cuối.

Đóng góp thứ tư là khung đánh giá ba mode trên cùng Base retriever, tách chất lượng retrieval, chất lượng answer và chi phí. Bên cạnh RAGAS và các metric xếp hạng, khóa luận sử dụng Legal Completeness Rate để đo mức bao phủ required facts đặc thù pháp lý. Bộ testset được mô tả và kiểm định theo hai lớp gồm quy tắc xác định và sáu tiêu chí LLM-as-a-judge; các trường hợp bị gán cờ được công khai để rà soát. Các số liệu chưa thể tái tạo từ output hiện có được gắn nhãn “nguồn báo cáo”, tránh trộn với kết quả đã tái tính.

1.7. Bố cục khóa luận

Chương 2 trình bày cơ sở lý thuyết và nghiên cứu liên quan. Chương 3 phân tích yêu cầu, dữ liệu và thiết kế kiến trúc. Chương 4 mô tả cài đặt, cấu hình và quy trình thực nghiệm. Chương 5 báo cáo kết quả, trả lời các câu hỏi nghiên cứu và phân tích giới hạn. Chương 6 tổng kết đóng góp và đề xuất hướng phát triển.

2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1. Retrieval-Augmented Generation

RAG gồm hai thành phần chính: retriever xác định tập tài liệu có khả năng liên quan và generator sinh chuỗi đầu ra dựa trên câu hỏi cùng tài liệu truy xuất [1]. Với truy vấn x , chuỗi trả lời y và tài liệu ẩn z , xác suất sinh có thể được mô tả bằng phép lấy tổng trên các tài liệu ứng viên:

$$p(y/x) = \sum_{z \in Z} p_{\eta}(z/x) p_{\theta}(y/x, z) \quad (1)$$

Trong đó, $p_{\eta}(z/x)$ là phân phối của retriever và $p_{\theta}(y/x, z)$ là xác suất của generator. Công thức cho thấy chất lượng đầu ra phụ thuộc đồng thời vào khả năng lấy đúng bằng chứng và khả năng sử dụng bằng chứng của mô hình sinh. Nếu tài liệu cần thiết không xuất hiện trong tập truy xuất, generator không thể tự động khôi phục căn cứ chỉ bằng prompt. Nếu tập truy xuất quá rộng, thông tin đúng có thể bị pha loãng bởi nội dung ít liên quan.

Trong khóa luận, RAG được dùng theo nghĩa kiến trúc hệ thống chứ không sao chép nguyên cấu hình huấn luyện của Lewis và cộng sự. Retriever là hệ thống hybrid Qdrant; generator là Qwen2.5-7B-Instruct chạy qua giao diện tương thích OpenAI/Ollama; kho tri thức là corpus văn bản pháp luật Việt Nam.

2.2. Biểu diễn dense và cosine similarity

Dense retrieval ánh xạ câu hỏi và đoạn văn thành vector liên tục. Các vector gần nhau biểu thị nội dung có quan hệ ngữ nghĩa. Với vector truy vấn q và vector chunk d , cosine similarity được tính như sau [6]:

$$\text{cosine}(q, d) = \frac{q^T d}{\|q\|_2 \|d\|_2} \quad (2)$$

Trong cấu hình chính đã xác minh, embedding được chuẩn hóa L2, Qdrant dùng khoảng cách cosine và mô hình dense là AITeamVN/Vietnamese_Embedding_v2 với vector 1.024 chiều. Hai biến thể fine-tuned và GTE từng xuất hiện trong các bản paper cũ không được dùng làm trục của khóa luận: workspace không có checkpoint fine-tuned, trong khi collection mang tên GTE vẫn khai báo và lưu vector 1.024 chiều dù cấu hình chính thức của gte-multilingual-base cho biết đầu ra 768 chiều [7], [15]. Vì vậy, số liệu lịch sử của hai biến thể này không được diễn giải như kết quả có thể tái lập.

2.3. Truy xuất thưa bằng BM25

Dense retrieval phù hợp với tương đồng ngữ nghĩa, còn BM25 hữu ích khi câu hỏi chứa số Điều, ký hiệu văn bản, thuật ngữ hoặc cụm từ pháp lý chính xác. Với từ t , tài liệu d , tần suất $f(t, d)$, độ dài $|d|$, độ dài trung bình $avgdl$ và các tham số k_1 , b , điểm BM25 có dạng [6]:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{avgdl} \right)} \quad (3)$$

Module BM25SparseVectorizer dùng token chữ thường sau chuẩn hóa Unicode NFKC, $k_1 = 1,5$, $b = 0,75$ và IDF làm trơn. Vector query giữ các term phù hợp để tích vô hướng sparse trong Qdrant tương ứng với điểm BM25 đã lưu ở phía corpus.

2.4. Reciprocal Rank Fusion

Kết quả dense và sparse có thang điểm khác nhau. Reciprocal Rank Fusion (RRF) kết hợp thứ hạng thay vì cộng trực tiếp điểm. Với tài liệu d , tập danh sách xếp hạng L và hằng số k_{RRF} , điểm hợp nhất là:

$$RRF(d) = \sum_{L \in L} \frac{1}{k_{RRF} + rank_L(d)} \quad (4)$$

Trong pipeline, Qdrant chạy dense search và BM25 sparse search song song trên collection child chunk, sau đó thực hiện fusion trước khi trả về top-k. Cách kết hợp này cho phép một chunk đạt thứ hạng cao nhờ tương đồng ngữ nghĩa, khớp thuật ngữ pháp lý, hoặc cả hai.

2.5. Chunking theo cấu trúc Điều - Khoản - Điểm

Chunking theo số token cố định có thể cắt giữa một Khoản hoặc tách điều kiện khỏi hệ quả pháp lý. Bộ VBPLChunker dùng biểu thức chính quy để nhận diện tiêu đề Điều, Khoản đánh số và Điểm ký hiệu chữ cái. Child chunk có thể là lời dẫn Điều, nội dung Khoản hoặc nội dung Điểm. Khi một child quá dài, hệ thống tách tiếp theo đoạn, câu và cuối cùng theo từ, mặc định không chồng lấn.

Mỗi child được gán `diều_id`, `parent_id`, loại cấu trúc, vị trí và tiền tố pháp lý. Định danh tổng quát là `<doc_id>_D<article>;` Khoản và Điểm thêm hậu tố `_K<clause>` và `_P<point>`; phân bị tách thêm `_S<n>`. Các child cùng `diều_id` được nối theo thứ tự nguồn thành parent chunk đại diện toàn văn Điều. Cấu trúc hai cấp cho phép mode naive dùng các phần nhỏ, còn mode `article_expand` và `critic` lấy toàn văn Điều khi cần.

Bảng 1. Cấu trúc định danh chunk và vai trò trong pipeline

Thành phần	Ví dụ định danh	Nội dung	Vai trò
Điều	..._D98	Nút cấu trúc của Điều 98	Liên kết parent và graph
Khoản	..._D98_K1	Một Khoản độc lập	Child retrieval
Điểm	..._D98_K1_Pa	Điểm a của Khoản 1	Child retrieval chi tiết
Parent	..._D98_PARENT	Toàn văn Điều 98	Mở rộng ngữ cảnh
Split	..._S2	Phần thứ hai của child dài	Giữ giới hạn đầu vào embedding

2.6. Đồ thị tri thức pháp luật

Một đồ thị tri thức có thể biểu diễn bằng bộ ba:

$$G = (V, E, R) \quad (5)$$

Trong đó, V là tập nút thực thể, E là tập cạnh và R là tập loại quan hệ. Trong đề tài, các loại nút chính gồm Điều, Khoản, Điểm, HànhVi, CheTai, ChuThe, NghĩaVu, QuyenHan, KhaiNiem và DieuKien. Quan hệ cấu trúc gồm `CO_KHOAN`, `CO_DIEM`; quan hệ ngữ nghĩa gồm `QUY_DINH_HANH_VI`, `CHE_TAI_CHINH`, `CHE_TAI_BO_SUNG` và `THAM_CHIEU`.

Đồ thị không được dùng như một nguồn thay thế văn bản gốc. Nó là chỉ mục cấu trúc giúp Critic phát hiện nơi có khả năng thiếu. Nội dung cuối cùng đưa vào generator vẫn là parent Article lấy từ Qdrant. Ranh giới này giảm rủi ro biến output trích xuất của LLM thành căn cứ pháp lý không thể kiểm tra.

2.7. Critic Agent và bổ sung bằng chứng có chọn lọc

Critic Agent hoạt động sau lần sinh nháp. Trước hết, các Điều trong top-k được lọc bằng tỷ lệ so với điểm Điều cao nhất và giới hạn số Điều cần kiểm tra. Critic truy vấn Neo4j để tìm ba tín hiệu:

- Một hành vi trong Điều có đồng thời CHE_TAI_CHINH và CHE_TAI_BO_SUNG.
- Số thành phần Khoản/Điều đã truy xuất nhỏ hơn tổng số thành phần của Điều.
- Có cạnh THAM_CHIEU đi ra hoặc đi vào giữa Điều đang xét và Điều chưa có trong ngữ cảnh.

Với mỗi ứng viên, công liên quan ngữ nghĩa yêu cầu mô hình trả lời nhị phân yes/no đối với câu hỏi và đoạn ứng viên. Chỉ ứng viên được chấp nhận mới được đưa vào phần ngữ cảnh bổ sung. Nếu danh sách chấp nhận rỗng, hệ thống giữ câu trả lời nháp; nếu có bằng chứng mới, hệ thống sinh lại từ đầu bằng prompt chung. Cơ chế này tách hai câu hỏi: cấu trúc cho biết “có thể thiếu ở đâu”, còn semantic gate quyết định “phần đó có cần cho câu hỏi hiện tại hay không”.

2.8. Nghiên cứu liên quan về legal QA và RAG

Kien và cộng sự xây dựng mô hình truy xuất Điều luật cho câu hỏi pháp lý tiếng Việt trên 5.922 câu hỏi, sử dụng biểu diễn neural attentive và đánh giá bằng recall, nDCG [2]. Nghiên cứu này chứng minh tính khả thi của Article retrieval nhưng không xử lý trực tiếp sinh câu trả lời hoặc phát hiện căn cứ thiếu sau truy xuất. Khazaeli và cộng sự kết hợp sparse search, embedding và reranking trong hệ thống legal QA định dạng tự do [3]. Đây là tiền đề gần với hybrid retrieval của khóa luận.

LegalBench và LawBench cho thấy năng lực pháp lý của mô hình cần được đánh giá trên nhiều nhóm tác vụ, không thể rút gọn thành một điểm duy nhất [8], [9]. KoBLEX đánh giá hỏi đáp pháp luật có hỗ trợ điều khoản và suy luận nhiều bước [10]. CLERC liên kết truy xuất án lệ với sinh phân tích tăng cường truy xuất [11]. Các bộ dữ liệu này khác hệ thống pháp luật và ngôn ngữ của đề tài, nhưng củng cố nhu cầu tách retrieval, evidence grounding và reasoning.

HippoRAG dùng đồ thị để hỗ trợ truy xuất dài hạn và liên kết thông tin qua nhiều đoạn [5]. Khóa luận chia sẻ trực giác rằng quan hệ giữa các đoạn có thể bổ sung cho similarity search, nhưng graph ở đây được thiết kế theo cấu trúc văn bản pháp luật Việt Nam và phục vụ missing-evidence critique, không nhằm tái hiện toàn bộ HippoRAG.

RAGAS tách các chiều context precision, faithfulness, answer relevancy và answer correctness [12]. ARES cũng xem xét context relevance, answer faithfulness và answer relevance [13]. Hai hướng đánh giá nhấn mạnh rằng một hệ thống RAG có thể truy xuất tốt nhưng sinh chưa faithful, hoặc sinh trôi chảy nhưng context chưa chính xác. Đề tài bổ sung LCR để đo mức bao phủ các facts pháp lý bắt buộc.

2.9. Chỉ tiêu đánh giá

2.9.1. Legal Completeness Rate

Với câu hỏi i , gọi F_i là tập required facts và $J(a_i, f)$ là phán đoán nhị phân của judge đối với câu trả lời a_i và fact f . Độ bao phủ câu hỏi là:

$$C_i = \frac{1}{|F_i|} \sum_{f \in F_i} 1[J(a_i, f) = 1] \quad (6)$$

LCR trên N câu hỏi là:

$$LCR = \frac{100}{N} \sum_{i=1}^N C_i \quad (7)$$

LCR thuộc khoảng $[0, 100]\%$. Judge dùng GPT-4o-mini ở nhiệt độ 0, tách biệt với generator Qwen. Prompt chấp nhận cách diễn đạt tương đương nhưng yêu cầu đúng con số và chi tiết pháp lý. Do LLM judge có thể chịu thiên lệch vị trí, độ dài và xu hướng tự ưu tiên [14], LCR cần được diễn giải cùng giới hạn chưa có validation với chuyên gia.

2.9.2. MRR và nDCG

Với hạng của kết quả liên quan đầu tiên r_i , truncated MRR@K là:

$$MRR@K = \frac{1}{N} \sum_{i=1}^N \frac{1[r_i \leq K]}{r_i} \quad (8)$$

nDCG@5 đánh giá chất lượng thứ hạng trong năm vị trí đầu:

$$nDCG@5 = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{r=1}^5 (2^{rel_r} - 1) / \log_2(r+1)}{IDCG_i@5} \quad (9)$$

Hai chỉ số thuộc $[0, 1]$; giá trị cao hơn tương ứng kết quả liên quan xuất hiện sớm và thứ tự top-5 tốt hơn [6].

2.9.3. Precision và recall theo Điều

Với tập Điều trong ngữ cảnh cuối A_i và tập Điều tham chiếu G_i :

$$P_i = \frac{|A_i \cap G_i|}{|A_i|}, R_i = \frac{|A_i \cap G_i|}{|G_i|} \quad (10)$$

Precision cho biết tỷ lệ Điều đã lấy là đúng, còn recall cho biết tỷ lệ Điều đúng đã được bao phủ. Metric này khác Context Precision của RAGAS: một metric hoạt động ở cấp tập ID Điều, metric còn lại đánh giá mức tập trung của các context đối với câu hỏi.

2.9.4. Chỉ số tài nguyên

Average Context Characters là số ký tự trung bình trong ngữ cảnh cuối. Pipeline Tokens per Question tính token trên toàn bộ lượt gọi trong pipeline, trong khi Final-call Tokens per Question chỉ tính lượt sinh câu trả lời cuối. LLM Calls per Question đếm số lần gọi mô hình. Các metric này không phải chi phí tiền tệ hay độ trễ; chúng chỉ mô tả lượng tài nguyên đã ghi trong kết quả tổng hợp.

2.10. Khoảng trống nghiên cứu

Các nghiên cứu trước cung cấp nền tảng về RAG, legal retrieval, multi-hop QA và đánh giá tự động, nhưng workspace chưa có một phương pháp trực tiếp so sánh ba chiến lược: giữ top-k chunk, mở rộng toàn Điều, và phát hiện thiếu bằng đồ thị rồi bổ sung có chọn lọc. Khoảng trống chính của đề tài là cơ chế kiểm tra sau retrieval dựa trên cấu trúc pháp lý, nhằm cải thiện đầy đủ mà không mặc định tăng toàn bộ context.

3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu chức năng

Hệ thống được thiết kế cho hai luồng chính: xây dựng dữ liệu offline và trả lời câu hỏi online. Luồng offline phải đọc văn bản DOCX, tách cấu trúc, tạo embedding, xây chỉ mục sparse/dense, trích xuất graph và nạp dữ liệu vào Qdrant/Neo4j. Luồng online phải nhận câu hỏi, truy xuất, chạy một trong ba mode, sinh câu trả lời và ghi lại các artefact cần cho đánh giá.

Các yêu cầu chức năng cụ thể gồm:

- Nhận dạng Điều, Khoản, Điểm và bảo toàn thứ tự nguồn.
- Tạo parent/child chunks với định danh ổn định.
- Hỗ trợ dense search, sparse search và RRF.
- Ánh xạ child hit về parent Điều.
- Truy vấn graph cho chế tài kép, thiếu cấu trúc và tham chiếu hai chiều.
- Lọc candidate theo điểm và semantic relevance.
- Chạy độc lập mode naive, article_expand, critic trên cùng input và Base retriever.
- Trả hoặc lưu answer, context, draft, critic report, token và số lần gọi LLM theo khả năng của từng entry point.
- Hỗ trợ resume khi đánh giá theo batch bị gián đoạn.

Người dùng mục tiêu ở phạm vi nguyên mẫu gồm: người tra cứu đặt câu hỏi pháp luật bằng tiếng Việt; người nghiên cứu so sánh ba mode; người quản trị cập nhật corpus, chỉ mục và đồ thị. Người tra cứu không phải biết cú pháp Điều/Khoản/Điểm, nhưng câu hỏi nên mô tả đủ chủ thể, hành vi, điều kiện và thông tin cần biết. Router được dùng trong phiên chat để tách hội thoại thông thường khỏi câu hỏi pháp luật; script đánh giá đặt skip_router=True để ba mode được chấm trên cùng câu hỏi và không cộng chi phí router vào số lượt gọi.

Bảng 2. Yêu cầu người dùng và tiêu chí chấp nhận

Mã	Vai trò/nhu cầu	Hành vi hệ thống	Tiêu chí chấp nhận
UR-01	Người tra cứu hỏi bằng tiếng Việt tự nhiên	Router phân loại legal hoặc chit_chat	Câu pháp luật đi vào retrieval; câu chào hỏi không truy xuất corpus
UR-02	Cần câu trả lời có căn cứ	Trả lời từ context đã lấy, kèm Điều/Khoản/Điểm khi có	Không viện dẫn ngoài context; cảnh báo nếu bằng chứng không đủ
UR-03	Câu hỏi có nhiều chế tài/cấu trúc	Cho phép chọn article_expand hoặc critic	Có thể khôi phục parent Điều và ghi danh sách Điều đã dùng
UR-04	Câu hỏi có dẫn chiếu	Critic kiểm tra cạnh THAM_CHIEU và semantic gate	Candidate vào final context khi gate trả yes; exception hiện hành là fail-open và phải được ghi nhận như rủi ro
UR-05	Người nghiên cứu cần so sánh	Chạy naive, article_expand, critic trên cùng query	--compare-all tạo ba kết quả có cùng đầu vào và cấu hình retrieval

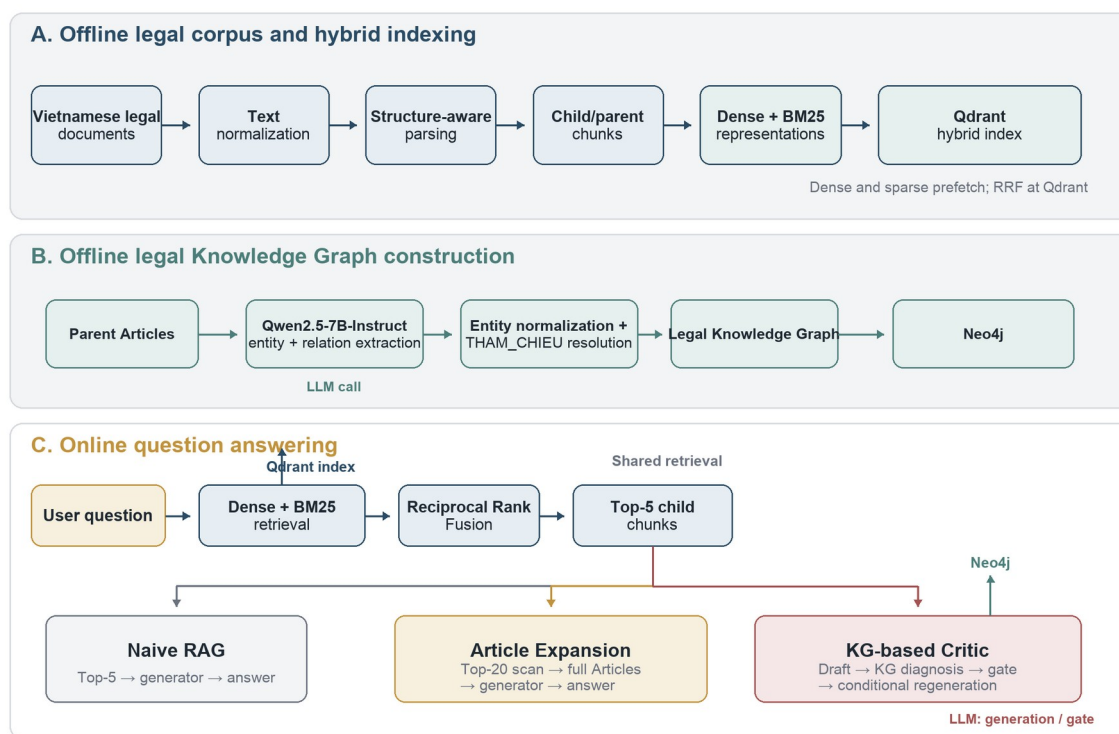
Mã	Vai trò/nhu cầu	Hành vi hệ thống	Tiêu chí chấp nhận
UR-06	Người quản trị cập nhật dữ liệu	Re-ingest Qdrant và xây lại Neo4j theo cùng snapshot	Collection, graph, model dimension và corpus version khớp nhau
UR-07	Cần truy vết thực nghiệm	Lưu context, draft, final, critic report và token	Script đánh giá có schema lưu theo record; UI hiện chưa hiển thị mọi trường trung gian

3.2. Yêu cầu phi chức năng

Tính truy vết được ưu tiên: mỗi chunk cần chứa vị trí pháp lý và ID nguồn; câu trả lời cần có context để kiểm tra. Tính tái lập yêu cầu lưu cấu hình, checkpoint, hash dữ liệu và output theo từng câu hỏi. Tính an toàn yêu cầu khóa truy cập Qdrant, Neo4j và API không nằm trong mã nguồn; .gitignore loại .env, API key và dữ liệu index khỏi commit.

Hệ thống chưa phải công cụ tư vấn pháp lý. Giao diện phải có cảnh báo phạm vi, ngày hiệu lực và yêu cầu đối chiếu văn bản gốc. Dữ liệu luật thay đổi theo thời gian nên mỗi văn bản cần metadata hiệu lực, nguồn phát hành, ngày thu thập và phiên bản hợp nhất.

3.3. Kiến trúc tổng thể

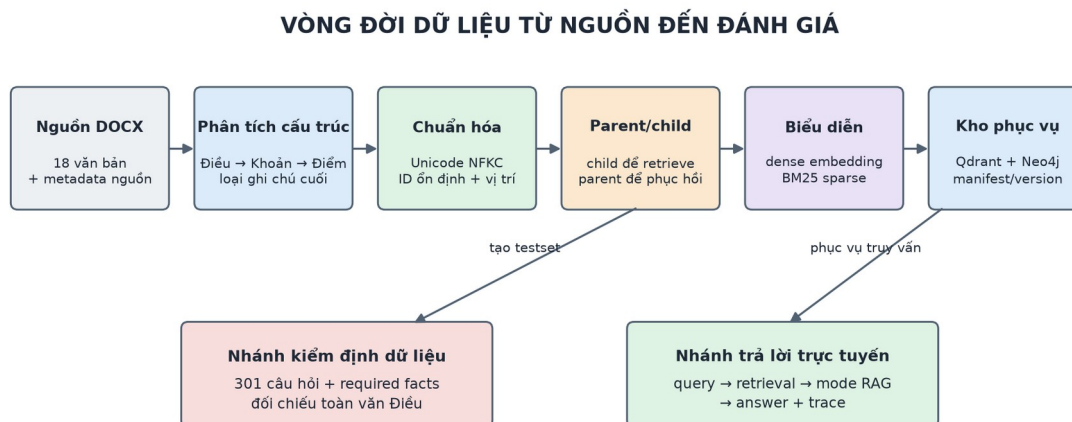


Hình 1. Kiến trúc offline và online với Base hybrid retriever dùng chung cho ba chế độ

Kiến trúc gồm hai phần. Ở phía offline, VBPLChunker tạo child/parent chunks; embedding và BM25 tạo hai loại vector để nạp Qdrant; Qwen2.5-7B-Instruct thực hiện trích xuất entity/relation, sau đó graph builder chuẩn hóa và Neo4j ingestor ghi đồ thị. Ở phía online, một truy vấn đi qua hybrid retrieval dùng chung. Từ đây pipeline rẽ sang ba mode. Naive sinh trực tiếp từ top-5 child. Article expansion khôi phục toàn Điều của các candidate được chọn. Critic sinh nháp, kiểm tra graph, lọc ứng viên và sinh lại có điều kiện.

Sự tách biệt offline/online có hai lợi ích. Thứ nhất, chi phí embedding và trích xuất graph được thực hiện trước, không làm tăng độ trễ của mọi câu hỏi. Thứ hai, nội dung văn bản và graph có thể được version độc lập với mô hình sinh. Tuy nhiên, nếu corpus thay đổi thì Qdrant và Neo4j phải được xây lại đồng bộ; nếu chỉ cập nhật một phía, Critic có thể tham chiếu ID không còn tồn tại.

3.4. Thiết kế dữ liệu văn bản



Hình 2. Vòng đời dữ liệu từ văn bản nguồn đến chỉ mục, đồ thị, testset và trả lời trực tuyến

Luồng dữ liệu bắt đầu từ văn bản DOCX, đi qua cùng một bộ phân tích cấu trúc để tránh chênh lệch giữa dữ liệu index và dữ liệu dùng kiểm định. Đầu ra của chunker tách thành ba nhánh có liên hệ bằng ID: child/parent phục vụ Qdrant; parent Article phục vụ trích xuất Neo4j; và toàn văn Điều phục vụ đối chiếu điều_ids của testset. Mỗi lần cập nhật corpus cần gắn version cho cả ba nhánh. Nếu chỉ tái tạo vector nhưng giữ graph hoặc testset theo snapshot cũ, các ID vẫn có thể hợp lệ về cú pháp nhưng sai về nội dung và hiệu lực.

3.4.1. Tiền xử lý

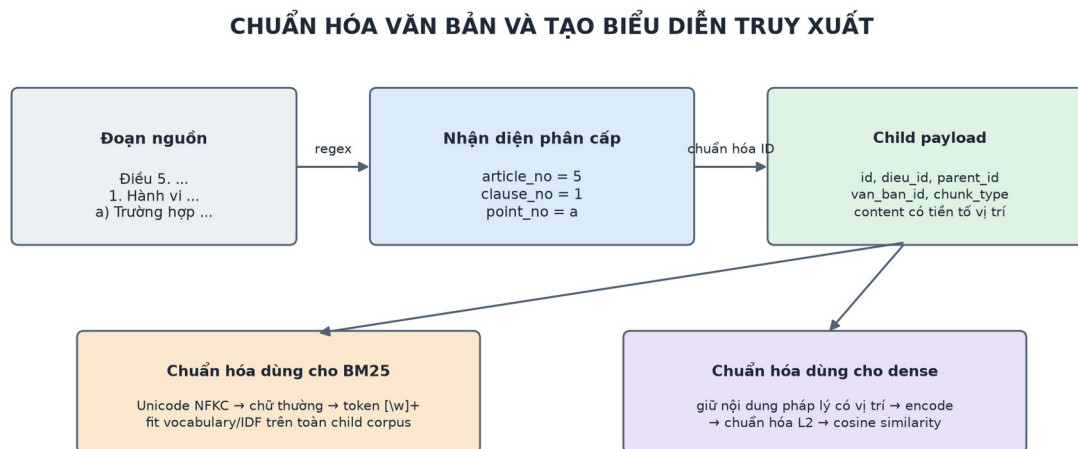
Chunker đọc các đoạn theo thứ tự xuất hiện trong DOCX, loại đoạn rỗng và dừng tại vùng ghi chú cuối văn bản đã nhận dạng. Tên file được chuẩn hóa thành `van_ban_id`; bộ regex nhận diện tiêu đề Điều <số>, Khoản dạng số và Điểm dạng chữ cái. Trạng thái phân cấp hiện hành được cập nhật khi gặp tiêu đề mới để mỗi đoạn nội dung mang đúng vị trí pháp lý. Một lời dẫn Điều chỉ chứa tiêu đề được ghép vào child nội dung đầu tiên khi có thể; nếu không, nó trở thành preamble child riêng. Với đoạn vượt giới hạn, thứ tự ưu tiên tách là đoạn văn, câu rồi từ; hậu tố split được thêm vào ID để không mất khả năng truy vết.

3.4.2. Cơ chế parent/child

Child chunk là đơn vị truy xuất vì tập trung vào một lời dẫn, Khoản hoặc Điểm, giảm nguy cơ một embedding phải đại diện cho nhiều chủ đề. Trước khi encode, nội dung child được gắn tiền tố vị trí gồm văn bản, Điều, Khoản và Điểm; cùng nội dung như “phạt tiền” nhờ đó có ngữ cảnh pháp lý phân biệt. Parent chunk là toàn văn một Điều theo đúng thứ tự nguồn và là đơn vị khôi phục ngữ cảnh. Mọi child có `parent_id` trỏ tới parent tương ứng. Với article expansion, hệ thống lấy `parent_id` từ child hit; với critic, graph đề xuất `diu_id` cần bổ sung nhưng văn bản bằng chứng vẫn được đọc từ parent collection, không sinh trực tiếp từ nội dung graph.

3.4.3. Kiểm soát trùng và va chạm ID

Các văn bản có thể lặp số Điều trong phụ lục hoặc bản hợp nhất. Hậu tố `_O<n>` được dùng cho occurrence trùng số; `_C<n>` xử lý collision của child; `_S<n>` đánh dấu split. Cơ chế này tránh ghi đè trong Qdrant và giữ khả năng truy ngược về đoạn nguồn.



Hình 3. Chuẩn hóa cấu trúc và biểu diễn truy xuất dense/BM25

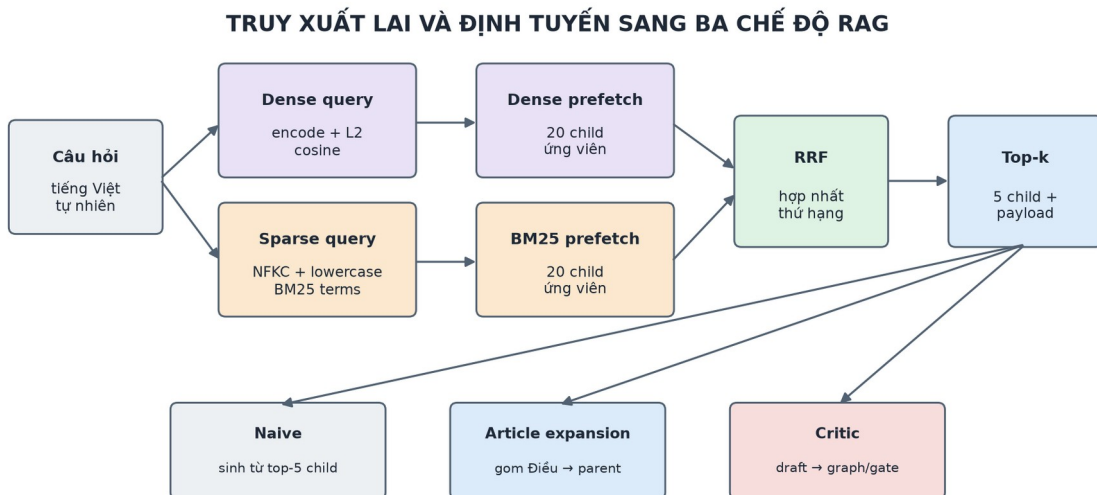
Chuẩn hóa không đồng nghĩa với xóa dấu hoặc rút gọn nội dung pháp luật. Phần văn bản cung cấp cho dense encoder được giữ nguyên ý nghĩa và thêm vị trí pháp lý; chỉ nhánh BM25 áp dụng Unicode NFKC, chuyển chữ thường và tách token bằng biểu thức `[w]+`. Vocabulary và IDF phải được fit một lần trên toàn child corpus rồi tái sử dụng ở thời điểm query. Dense vector được chuẩn hóa L2 trước khi tìm kiếm cosine. Hai nhánh do đó có phép chuẩn hóa khác nhau nhưng cùng tham chiếu một child_id và payload.

3.5. Thiết kế chỉ mục Qdrant

Collection `legal_child_chunks` lưu named vectors dense và bm25, cùng payload gồm ID, nội dung, cấu trúc và parent. Collection `legal_parent_chunks` lưu toàn văn từng Điều. Chỉ mục Base hiện có 10.355 child points, 1.211 parent points và dense cosine 1.024 chiều. BM25 index được fit trên toàn bộ child corpus với `doc_count=10.355`, `vocab_size=1.914`, `avgdl≈57,5987`, $k_1=1,5$ và $b=0,75$, rồi lưu để query encoder dùng đúng vocabulary và IDF.

Artefact Base có hash metadata `8a8248...91a5`; tệp BM25 có hash `59ed...18d1`. Các chuỗi rút gọn này được dùng để nhận diện snapshot, không thay cho manifest bất biến đầy đủ. Collection GTE hiện hữu bị loại khỏi đánh giá chính vì kích thước 1.024 không khớp 768 chiều của model card [15]; dùng collection đó để suy luận kết quả GTE sẽ không bảo đảm đúng mô hình.

Ở truy vấn, model embedding và BM25 vectorizer được load một lần khi khởi tạo pipeline. Chuỗi xử lý gồm: (1) nhận nguyên văn câu hỏi; (2) dense encoder tạo vector chuẩn hóa L2; (3) BM25 vectorizer áp dụng đúng pipeline NFKC/chữ thường/token và vocabulary đã fit; (4) Qdrant chạy hai prefetch độc lập, mặc định mỗi nhánh lấy 20 candidate; (5) RRF hợp nhất theo thứ hạng; (6) trả top-5 child cùng payload; và (7) ánh xạ parent_id/dieu_id cho mode downstream. Việc tái sử dụng model, client và BM25 giúp tránh thời gian load lại cho mỗi câu hỏi.



Hình 4. Pipeline truy xuất lai dense/BM25, RRF và ba chế độ RAG

Bảng 3. Các bước xử lý một truy vấn từ đầu vào đến ngữ cảnh sinh

Bước	Đầu vào	Xử lý	Đầu ra/kiểm soát
1	Chuỗi người dùng	Loại input rỗng; router phân loại hội thoại/pháp luật	chit_chat trả trực tiếp; legal đi tiếp
2	Câu hỏi pháp luật	Dense encode bằng Base và chuẩn hóa L2	Vector 1.024 chiều đúng collection
3	Cùng câu hỏi	NFKC, lowercase, token hóa BM25	Sparse indices/values theo vocabulary corpus
4	Hai vector query	Prefetch dense và sparse trên child collection	Hai danh sách tối đa 20 candidate
5	Hai bảng hạng	Qdrant FusionQuery RRF	Top-5 child đã hợp nhất
6	Child payload	Lấy dieu_id, parent_id, score và content	Context cơ sở, danh sách Điều truy xuất
7	Mode	Giữ child, mở rộng parent hoặc critic graph/gate	Final context và answer có trace

Bảng 4. Payload tối thiểu của child và parent chunk

Trường	Child	Parent	Ý nghĩa
id	Có	Có	Định danh duy nhất
dieu_id	Có	Có	Khóa nhóm theo Điều
parent_id	Có	Không bắt buộc	Liên kết child-parent
van_ban_id	Có	Có	Định danh văn bản
chunk_type	Có	Parent	Loại cấu trúc
content	Có	Có	Nội dung truy xuất hoặc toàn Điều
dense	Có	Tùy cấu hình	Vector ngữ nghĩa
bm25	Có	Không	Vector thưa

3.6. Thiết kế Knowledge Graph

3.6.1. Trích xuất hai lượt

Parent Article được đưa vào Qwen2.5-7B-Instruct. Pass 1 trích xuất entity với giới hạn 2.048 token; Pass 2 nhận danh sách ID entity rồi trích xuất relation với giới hạn 1.024 token. Nhiệt độ bằng 0 và tối đa ba lần thử. Model được lượng tử hóa NF4 4-bit để phù hợp GPU T4 16 GB.

Tách entity và relation làm hai lượt giảm kích thước JSON trong mỗi lần sinh và giảm trường hợp output bị cắt giữa chừng. Hàm sửa JSON tìm phần đóng ngoặc hợp lệ gần nhất để giữ phần dữ liệu đã hoàn thành. Đây là biện pháp phục hồi cú pháp, không bảo đảm nội dung được trích xuất là đúng.

3.6.2. Chuẩn hóa và placeholder

Graph builder chuyển ID sang dạng không dấu, chữ thường và gắn tiền tố văn bản. Nút và cạnh được deduplicate trước khi nạp. Với THAM_CHIEU, nếu đích chưa có, builder tạo placeholder is_placeholder=True. Placeholder cho phép graph duy trì thông tin “có một dẫn chiếu” nhưng cần được giải quyết trước khi dùng cho production.

3.6.3. Lược đồ graph

Bảng 5. Các nhóm nút và quan hệ chính trong đồ thị

Nhóm	Nhân/quan hệ	Mục đích
Cấu trúc	Dieu, Khoan, Diem	Bảo toàn phân cấp pháp luật
Nội dung	HanhVi, CheTai, ChuThe, NghiaVu	Mô tả đối tượng và quy tắc
Cạnh cấu trúc	CO_KHOAN, CO_DIEM	Duyệt thành phần cùng Điều
Cạnh chế tài	CHE_TAI_CHINH, CHE_TAI_BO_SUNG	Phát hiện chế tài kép
Cạnh dẫn chiếu	THAM_CHIEU	Tìm Điều liên quan ngoài top-k
Cạnh quy định	QUY_DINH_HANH_VI	Liên kết cấu trúc với hành vi

3.7. Thiết kế ba chế độ RAG

3.7.1. Naive RAG

Naive là baseline. Pipeline lấy top-5 child chunks và gọi generator bằng prompt chung. Mode này không khôi phục parent và không truy vấn graph. Ưu điểm là context nhỏ và luồng đơn giản; nhược điểm là dễ thiếu Khoản hoặc Điều liên quan nằm ngoài top-5.

3.7.2. Article expansion

Mode mở rộng quét pool tối đa 20 child candidates, gom score theo Điều bằng giá trị child cao nhất, giữ các Điều đạt ít nhất 40% score Điều đứng đầu, lấy tối đa năm Điều và sắp toàn văn giảm dần theo score. Mode không dùng graph và không theo THAM_CHIEU. Các giá trị 0,60 trong source snapshot cũ bị thay bởi default 0,40 của mã nguồn hiện hành.

3.7.3. KG-based critic

Critic bắt đầu từ top-5 child và câu trả lời nháp. Các Điều có score ít nhất 60% score cao nhất được đưa vào kiểm tra, tối đa bốn Điều. Engine so sánh số phần đã lấy với tổng phần theo Qdrant/Neo4j, tìm chế tài kép và duyệt THAM_CHIEU ra/vào; traversal nhiều bước giới hạn max_hops=2 và tổng số Điều lấy qua graph tối đa tám. Đối với cross-Article, gate đánh giá

parent ứng viên; với same-Article, gate đánh giá các anchor child và chấp nhận Điều nếu ít nhất một anchor liên quan. Candidate được chấp nhận đóng góp toàn parent Article.

Nếu có candidate, hệ thống nối chúng vào context gốc và sinh lại từ đầu. Nếu không, draft là final answer. Một safeguard riêng lập parent Article đứng đầu và sinh lại khi initial top-k chứa từ hai văn bản trở lên nhưng graph không lấy được Điều mới. Exception của gate hiện được xử lý theo fail-open: candidate được chấp nhận để thiên về recall. Đây là hành vi đã xác minh từ code, đồng thời là rủi ro vì lỗi kỹ thuật chưa được tách khỏi quyết định “yes”.

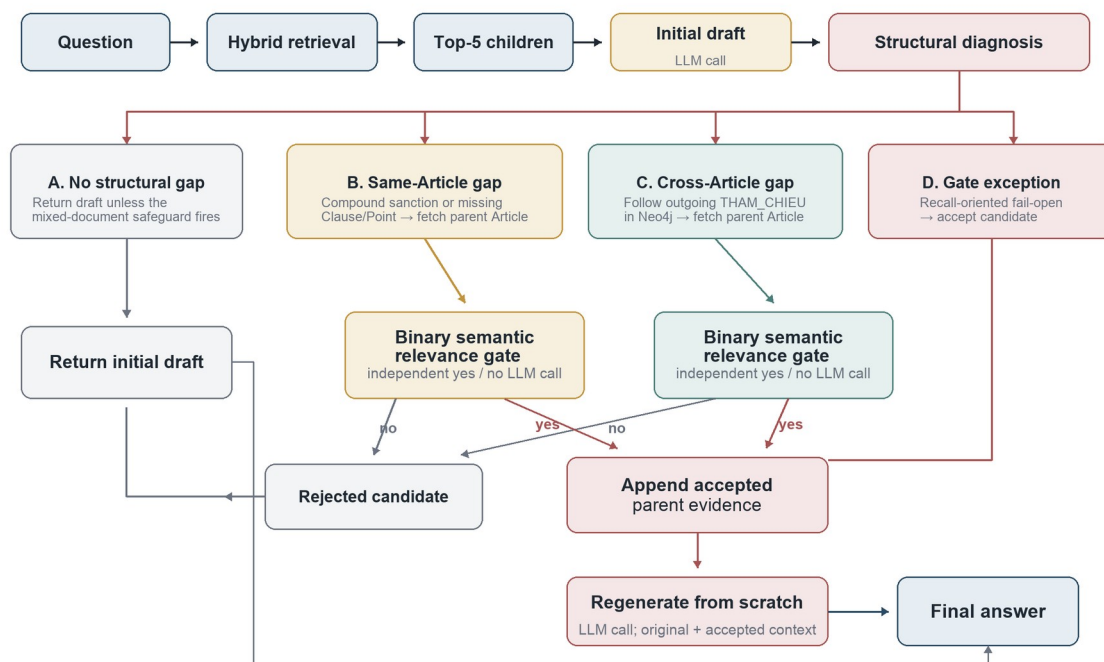
Bảng 6. So sánh vận hành của ba chế độ

Tham số	Naive RAG	Article expansion	KG-based critic
Candidate retrieval	Top-5 child	Top-20 child	Top-5 child
Số Điều tối đa	Không áp dụng	5	4 để kiểm tra
Ngưỡng tương đối	Không áp dụng	0,40	0,60
Toàn văn Điều	Không	Có	Có, khi được chấp nhận
Truy vấn Neo4j	Không	Không	Có
Semantic gate	Không	Không	Yes/No
Sinh nháp	Không tách	Không tách	Có
Sinh lại có điều kiện	Không	Không	Có

3.8. Thuật toán Critic Agent

Thuật toán được mô tả ở mức logic như sau:

1. Nhận truy vấn, top-k child, danh sách Điều và score.
2. Chọn tập focus theo tỷ lệ score và giới hạn critic_max_dieu.
3. Đếm số phần cấu trúc đã retrieve cho mỗi Điều.
4. Truy vấn graph để tìm chế tài kép, thiếu cấu trúc và dẫn chiếu chéo.
5. Hợp nhất các ID Điều cần bổ sung, loại ID đã có.
6. Lấy parent text từ Qdrant theo ánh xạ ID chuẩn.
7. Chạy semantic gate cho từng candidate.
8. Nếu không có candidate đạt, trả draft; nếu có, ghép context và regenerate.
9. Ghi critic_report, graph_fetched_dieu_ids, token và số lượt gọi.



Hình 5. Lưu đồ quyết định của Critic: phát hiện thiếu, semantic gate và sinh lại có điều kiện

Bảng quyết định sau phân biệt hành vi đã cài đặt và nhánh chỉ được đề xuất. Không phát hiện gap dẫn tới trả draft, trừ safeguard nhiều văn bản. Gap cùng Điều hoặc THAM_CHIEU tạo candidate parent; gate yes hoặc exception làm candidate được chấp nhận, gate no làm candidate bị loại và ID được ghi vào báo cáo. Khi toàn bộ candidate bị loại, hệ thống giữ draft. Query rỗng bị CLI bỏ qua; mode không hợp lệ bị argparse từ chối. Mã hiện hành có try/except cục bộ quanh gate và một số truy vấn graph, nhưng chưa có circuit breaker hoặc thông báo người dùng chuẩn hóa cho mọi lỗi Qdrant, Neo4j và LLM; các cơ chế đó là hướng phát triển, không phải chức năng đã hoàn tất.

Bảng 7. Bảng quyết định các nhánh vận hành đã xác minh và đề xuất

Tình huống	Hành vi hiện hành	Trạng thái
Không có evidence gap	Trả draft; có thể kích hoạt safeguard nếu top-k chứa nhiều văn bản	Đã xác minh từ code
Thiếu Khoản/Điểm, chế tài bổ sung hoặc khắc phục trong cùng Điều	Lấy parent Article, chạy gate, sinh lại nếu chấp nhận	Đã xác minh từ code
Có THAM_CHIEU sang Điều khác	Duyệt graph tối đa hai hop, lấy parent, gate từng candidate	Đã xác minh từ code
Gate trả yes	Thêm parent vào context được chấp nhận	Đã xác minh từ code
Gate trả no	Loại candidate, ghi ID bị từ chối	Đã xác minh từ code
Gate exception	Fail-open, chấp nhận candidate	Đã xác minh; rủi ro
Không có candidate được chấp nhận	Giữ draft	Đã xác minh từ code

Tình huống	Hành vi hiện hành	Trạng thái
Câu hỏi rỗng/mode sai ở CLI	Bỏ qua dòng rỗng/argparse từ chối mode	Đã xác minh từ code
Câu hỏi ngoài corpus	Có thể vẫn sinh từ context gần nhất; chưa có abstention chắc chắn	Giới hạn hiện tại
Mất Qdrant/Neo4j/LLM	Chưa có UX và policy nhất quán cho mọi điểm lỗi	Hướng phát triển

Thiết kế cố ý không cho graph tạo câu trả lời. Graph chỉ đề xuất vị trí thiếu và lý do; generator vẫn đọc văn bản đầy đủ. Điều này giúp lỗi relation của graph ít trực tiếp biến thành nội dung trả lời hơn, dù vẫn có thể làm hệ thống bỏ sót hoặc lấy thừa candidate.

3.9. Thiết kế bộ đánh giá

Bộ eval_testset.jsonl có 301 record, mỗi record gồm id, category, diều_ids, van_ban, question, required_facts và reference_answer. Bốn nhóm được thiết kế để tách các loại khó khăn: 51 câu chế tài kép trong cùng Điều, 16 câu tham chiếu chéo, 130 câu đa thành phần cấu trúc và 104 câu đối chứng không có khoảng trống.

Bảng 8. Phân bố bộ câu hỏi đánh giá

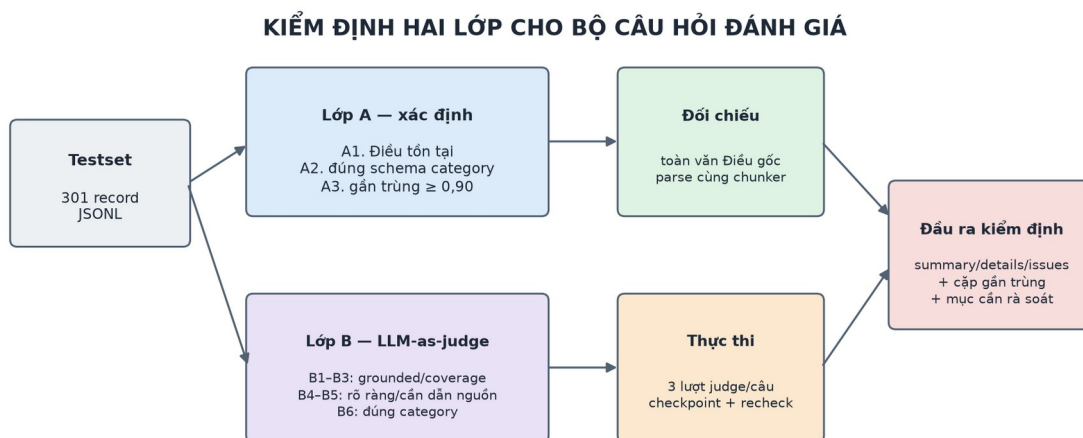
Nhóm	Số câu	Khó khăn cần đo
same_diều_compound_penalty	51	Phạt chính, bổ sung và khắc phục ở các phần khác nhau
cross_reference	16	Cần theo dẫn chiếu sang Điều khác
structural_multi_part	130	Cần nhiều Khoản/Điểm cùng Điều
control_no_gap	104	Ngữ cảnh top-k đã tương đối đầy đủ
Tổng	301	

Base retriever chạy cả ba mode, tạo ba cấu hình end-to-end. Kết quả per-question cần chứa answer, retrieved contexts, ranked Điều, Điều graph lấy thêm, draft và critic report. Artefact ngày 31/07/2026 có 301 record cho Naive và 301 record cho Article Expansion nhưng chỉ 23 record cho Critic; vì vậy, phần kết quả tách rõ metric có thể tái tính từ raw Naive với metric tổng hợp do FinalFix báo cáo, và không thực hiện kiểm định cặp.

3.9.1. Kiểm định xác định và đối chiếu toàn văn Điều

Nguồn testset_validation_methodology.md mô tả một quy trình kiểm định độc lập với bước chạy RAG. Để tránh “tự kiểm tra bằng chính dữ liệu đáp án”, mỗi diều_id được ánh xạ về toàn văn Điều gốc được parse bằng cùng VBPLChunker; judge nhận phần văn bản này cùng câu hỏi, required facts và reference answer. Lớp A không dùng mô hình ngôn ngữ: A1 kiểm tra mọi diều_id tồn tại; A2 kiểm tra nhóm cross_reference có ít nhất hai ID và ba nhóm còn lại có đúng một ID; A3 tìm câu hỏi gần trùng bằng difflib.SequenceMatcher với ngưỡng 0,90.

Lớp B dùng LLM-as-a-judge. Ba tiêu chí về tám nguồn được hỏi chung trong lượt thứ nhất, hai tiêu chí ngôn ngữ/citation trong lượt thứ hai và tiêu chí phân nhóm trong lượt thứ ba. Tách ba lượt giúp prompt ngắn hơn và cho phép chạy lại riêng B6 bằng --recheck-category; checkpoint được ghi sau mỗi câu để tiếp tục khi gián đoạn. Cách đánh giá tự động này phù hợp với hướng RAGAS, ARES và LLM-as-a-judge [12]–[14], nhưng vẫn cần đối chiếu người thật trước khi xem là xác nhận pháp lý.



Hình 6. Quy trình kiểm định hai lớp cho testset và các artefact đầu ra cần lưu

Bảng 9. Chín kiểm tra dùng để kiểm định bộ testset

Mã	Loại	Câu hỏi kiểm định	Điều kiện đạt
A1	Xác định	Mọi dieu_id có trong corpus?	Không có ID không ánh xạ được
A2	Xác định	Số Điều có phù hợp category?	Cross-reference ≥ 2 ; nhóm khác =1
A3	Xác định	Có câu hỏi gần trùng?	Gần cỡ cặp similarity $\geq 0,90$ để rà soát
B1	Judge	Required facts có bám toàn văn Điều?	Mỗi fact được văn bản hỗ trợ
B2	Judge	Reference answer có bám toàn văn Điều?	Không thêm kết luận không có căn cứ
B3	Judge	Đáp án có bao phủ required facts?	Không bỏ ý bắt buộc
B4	Judge	Câu hỏi có tự nhiên, rõ ràng?	Có thể hiểu mà không cần ngữ cảnh ẩn
B5	Judge	Câu hỏi có thật sự cần dẫn nguồn?	Không thể trả lời đáng tin chỉ bằng kiến thức chung
B6	Judge	Nhãn category có đúng cơ chế cần đo?	Phù hợp định nghĩa của một trong bốn nhóm

3.9.2. Chính sách diễn giải kết quả kiểm định

Kết quả lớp A có thể tái lập từ scripts/validate_testset.py, corpus và testset đúng snapshot; A1/A2 cho 0 lỗi và A3 gần cỡ hai cặp gần trùng. Kết quả lớp B còn phụ thuộc model, revision, prompt và retry policy. Workspace không có checkpoint hay các tệp summary/details/issues của lần chạy B1-B6, nên Chương 5 báo cáo tỷ lệ và ID đúng theo tài liệu nguồn, không tuyên bố đã chạy lại. Hai cặp gần trùng không tự động bị xóa: chúng chỉ bị loại nếu rà soát cho thấy cùng ý định và cùng bằng chứng.

3.10. Thiết kế bảo mật và quản trị dữ liệu

Neo4j password, API key và endpoint được đọc từ biến môi trường; giá trị REPLACE_ME chỉ là placeholder. Service account cần nguyên tắc quyền tối thiểu. Dữ liệu pháp luật là công

khai nhưng lịch sử truy vấn người dùng có thể phản ánh vấn đề pháp lý nhạy cảm; log production phải tách định danh, giới hạn retention và không dùng để huấn luyện nếu chưa có cơ sở pháp lý.

Quản trị corpus cần manifest chứa URL nguồn chính thức, ngày tải, ngày hiệu lực, trạng thái hết hiệu lực, hash file và phiên bản chunker. Audit trong workspace đã phát hiện nhiều văn bản cần thay thế hoặc cập nhật sau ngày 01/07/2026. Vì vậy, hệ thống phải hiển thị ngày chụp corpus và không diễn giải câu trả lời như quy định hiện hành nếu chưa kiểm tra phiên bản.

4. CÀI ĐẶT VÀ THỰC NGHIỆM

4.1. Tổ chức mã nguồn

Mã nguồn hiện hành được chia theo trách nhiệm: src/data_processing cho chunking; src/data_ingestion cho Qdrant; src/retrieval cho BM25 và hybrid search; src/knowledge_graph cho extractor, builder và Neo4j; src/agents cho các agent thành phần; src/workflow nối các node bằng state chung; scripts cho ingest, chatbot, đánh giá và kiểm định testset. Hai entry point sản phẩm là scripts/run_chatbot.py (CLI) và app.py (Chainlit).

Bảng 10. Các module chính của hệ thống

Module	Tệp tiêu biểu	Trách nhiệm
Data processing	src/data_processing/chunking.py	Điều - Khoản - Điểm, parent/child
Retrieval	bm25_sparse.py, qdrant_hybrid_search.py	Sparse/dense và RRF
KG extraction	entity_extractor.py, graph_builder.py	Entity, relation, chuẩn hóa
KG storage	neo4j_ingest.py	Nạp và truy vấn Neo4j
Agent/workflow	src/workflow/pipeline.py, src/agents/agent_critic/critic_query.py	Router, ba mode, completeness check và state
Product	scripts/run_chatbot.py, app.py	CLI, compare-all và giao diện Chainlit
Evaluation	run_evaluation.py, score_evaluation.py, validate_testset.py	Thu output, LCR/RAGAS, retrieval metrics và kiểm định benchmark

4.2. Môi trường và mô hình

Pipeline dùng Python, Qdrant embedded hoặc server, Neo4j và LLM qua endpoint tương thích OpenAI. CLI mặc định gọi qwen2.5:7b qua Ollama ở temperature 0,2. Chainlit mặc định chọn gpt-4o-mini qua proxy cấu hình trong code và còn cho phép Qwen2.5 7B, GPT-5-nano hoặc GPT-4.1-nano; việc tồn tại lựa chọn không chứng minh mọi model đã được dùng trong thí nghiệm. Graph extraction được source cũ mô tả dùng Qwen2.5-7B-Instruct; archive extraction hiện có nhưng môi trường và log trích xuất chưa đủ để chạy lại hoàn toàn.

Cấu hình chính của luận văn là Base retriever. Các tham số được đọc trực tiếp từ src/workflow/pipeline.py: top_k=5, prefetch_limit=20, article_expand_score_ratio=0,4, article_expand_max_dieu=5, critic_score_ratio=0,6, critic_max_dieu=4. Script đánh giá đặt skip_router=True; phiên chat tương tác giữ router.

Bảng 11. Cấu hình thực nghiệm chính

Thành phần	Giá trị
Dense model chính	AITeamVN/Vietnamese_Embedding_v2
Dense dimension	1.024, cosine, chuẩn hóa L2
Sparse model	BM25, $k_1 = 1,5$, $b = 0,75$
Fusion	RRF
Top-k child	5

Thành phần	Giá trị
Candidate pool của expand	20
Ngưỡng expand	0,40 của score cao nhất
Số Điều expand tối đa	5
Ngưỡng critic	0,60 của score cao nhất
Số Điều critic kiểm tra tối đa	4
Duyệt THAM_CHIEU	Tối đa 2 hop, tối đa 8 Điều fetch
Generator/gate CLI	qwen2.5:7b, temperature 0,2
Judge/RAGAS evaluator	Được paper/methodology báo cáo; chưa tái chạy đầy đủ

4.3. Corpus và chỉ mục

Repository hiện chứa 23 tài liệu trong data/keep: 18 DOCX và năm PDF. Chạy lại VBPLChunker trên chính 18 DOCX tạo 1.211 parent Articles và 10.355 child chunks, gồm 5.159 Khoản, 4.964 Điểm và 232 điều_preamble; có 1.211 parent_id duy nhất. Kết quả này đồng thời khớp số điểm trong hai collection Base. Con số 1.209 của manifest cũ bị loại khỏi mô tả chính vì không khớp phép đếm tái lập. Năm PDF chưa được parser xử lý nên không phải bằng chứng retrievable của các kết quả.

Quá trình ingest tạo parent points, fit BM25 trên toàn bộ child text, sau đó encode và upsert child theo batch. Tùy chọn --resume bỏ qua ID đã có; --recreate xóa và tạo lại collection, vì vậy phải chỉ rõ đường dẫn collection để tránh mất index khác. Metadata hash của Base và BM25 đã được ghi trong audit. Tài liệu gte_base.md vẫn hữu ích như hướng dẫn re-ingest đúng 768 chiều, nhưng collection GTE hiện tại không đáp ứng điều kiện đó và không được dùng trong kết quả chính.

[CHỜ ẢNH DEMO: demo_qdrant_collection.png]

4.4. Xây dựng và nạp đồ thị

Graph pipeline duyệt parent Article theo batch nhỏ để tránh hết bộ nhớ. Mỗi output extraction được lưu JSONL, cho phép debug và retry. Graph builder chuẩn hóa ID, loại node/cạnh trùng, sửa liên kết cross-document và tạo Cypher. Neo4j ingestor thực thi các batch query và tạo index/constraint khi cần.

Archive extraction cũ chứa 18 JSONL với 1.200 record, nhưng audit phát hiện một member Giao dịch điện tử bị trùng byte với member Viễn thông và chứa ID viễn thông. Neo4j không thể truy vấn trực tiếp trong phiên hoàn thiện vì Docker daemon không chạy. Do đó, các số 15.666 node, 14.137 relation, 1.612 THAM_CHIEU và 94,1% Article coverage chỉ được ghi là số liệu paper báo cáo, không phải số liệu đã tái kiểm; khóa luận không dùng chúng để suy luận định lượng về chất lượng graph.

[CHỜ ẢNH DEMO: demo_neo4j_tham_chieu.png]

4.5. Quy trình thao tác và các kịch bản đầu vào

Sản phẩm hiện có hai entry point đã xác minh. scripts/run_chatbot.py là CLI phục vụ nghiên cứu: nhận --query, --mode, --top-k, --compare-all hoặc mở vòng lặp tương tác. app.py là giao diện Chainlit thực, có ba Chat Profile, đăng nhập và lịch sử cục bộ, bộ chọn embedding/LLM và bảng Critic Report bên cạnh câu trả lời. Tuy nhiên, UI chưa hiển thị đồng thời mọi trường

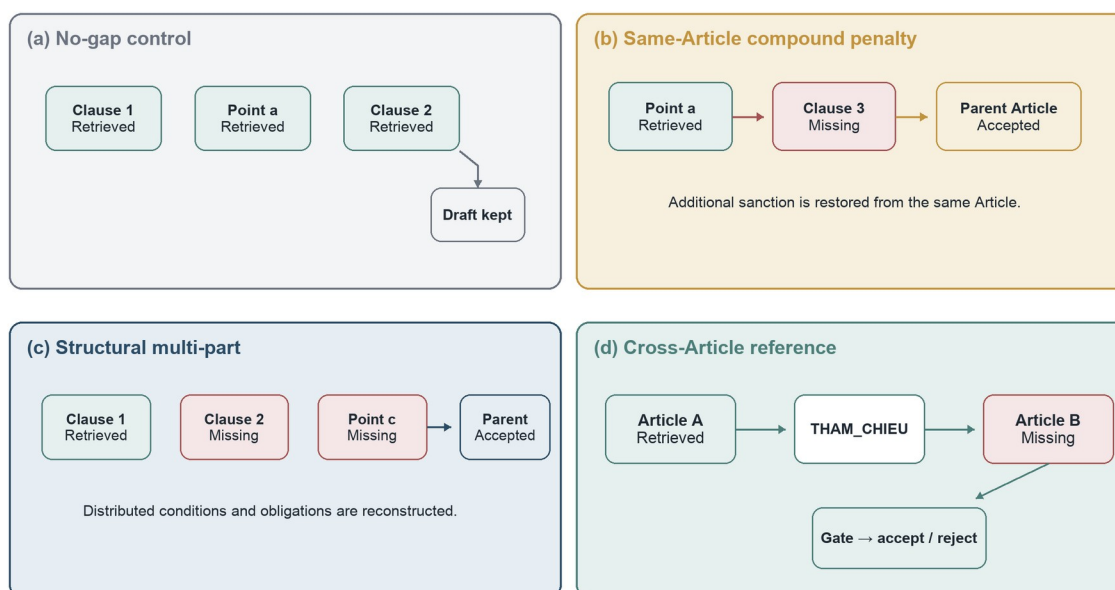
mà protocol demo yêu cầu, chẳng hạn toàn bộ top-k payload, raw gate decision, danh sách Điều fetch và token của từng lượt. Vì vậy, tài liệu gọi đây là nguyên mẫu CLI/Chainlit, không gọi là sản phẩm tư vấn pháp lý hoàn chỉnh.

Quy trình khởi động theo code hiện hành gồm: tạo .env từ mẫu và điền khóa nếu dùng model qua API; chạy docker compose up -d neo4j; nạp graph một lần bằng docker compose --profile ingest up kg-ingest; khởi động Chainlit bằng docker compose up -d app; sau đó mở http://localhost:8000. Ollama chỉ cần khi chọn Qwen local. Trong phiên kiểm toán, Docker daemon không hoạt động nên quy trình này chưa được chạy end-to-end và chưa có ảnh demo hợp lệ.

CLI Base được gọi rõ ràng bằng hai biến môi trường để tránh default fine-tuned đã lỗi thời trong docstring của script:

```
QDRANT_PATH=data/.qdrant_base
EMBEDDING_MODEL=AITeamVN/Vietnamese_Embedding_v2
python scripts/run_chatbot.py --query "..." --mode critic
```

Để so sánh cùng một câu hỏi: QDRANT_PATH=data/.qdrant_base
EMBEDDING_MODEL=AITeamVN/Vietnamese_Embedding_v2 python
scripts/run_chatbot.py --query "..." --compare-all. Script in câu hỏi, mode, số chunk, số Điều, draft và critic report; final answer luôn được in. Các trường graph_fetched_dieu_ids và token có trong state/output của pipeline và file đánh giá nhưng CLI chưa in đầy đủ. Đây là khác biệt giữa dữ liệu nội bộ và dữ liệu được trình bày trên giao diện.



Hình 7. Bốn kiểu khoảng trống bằng chứng và hành vi của Critic trong nguyên mẫu

[CHỜ ẢNH DEMO: demo_chainlit_overview.png]

[CHỜ ẢNH DEMO: demo_compare_all_cli.png]

[CHỜ ẢNH DEMO: demo_critic_trace_json.png]

Bảng 12. Các kịch bản nhập câu hỏi và hành vi mong đợi

Kịch bản	Ví dụ/mệnh lệnh	Mode phù hợp	Hành vi mong đợi
Hội thoại thông thường	“Xin chào”	Bất kỳ trong phiên chat	Router trả lời xã giao, is_chit_chat=True; evaluation bỏ router
Một quy định đơn lẻ	“Mức phạt đối với hành vi X là bao nhiêu?”	naive	Trả lời từ top-k child; nhanh, dùng ít context
Nhiều phần trong cùng Điều	“Ngoài phạt tiền còn đình chỉ hay khắc phục gì?”	article_expand	Gom candidate theo Điều và đưa toàn parent Article vào context
Dẫn chiếu/điều kiện phức hợp	“Trường hợp nào áp dụng ngoại lệ được dẫn chiếu tại Điều khác?”	critic	Sinh draft, kiểm tra graph, gate parent liên quan rồi mới sinh lại
So sánh thuật toán	--query "..." --compare-all	Cả ba	Chạy cùng input qua ba mode và in kết quả tách biệt
Nhập nhiều câu liên tiếp	Không truyền --query	Tương tác	Hiện prompt, bỏ qua dòng rỗng, tiếp tục đến khi người dùng thoát
Điều chỉnh độ rộng retrieval	--top-k 8	Mode đã chọn	Lấy số child theo tham số; log phải ghi top-k để tái lập
Input/mode không hợp lệ	Query rỗng hoặc mode ngoài enum	Không áp dụng	Dòng rỗng bị bỏ qua; argparse từ chối mode ngoài lựa chọn
Không có bằng chứng đủ	Câu hỏi ngoài corpus	Mode bất kỳ	Code chưa có abstention chắc chắn; cần coi câu trả lời là không đủ bằng chứng

Để đặt câu hỏi có chất lượng, người dùng nên nêu rõ chủ thể, hành vi, bối cảnh và thông tin cần biết. Chẳng hạn, “Doanh nghiệp bị xử lý thế nào khi thu thập dữ liệu cá nhân không có sự đồng ý, có bị đình chỉ không?” rõ hơn “Vi phạm dữ liệu bị sao?”. Dù vậy, code hiện chưa có nhánh hỏi lại có cấu trúc khi thiếu tình tiết; đây là yêu cầu tương lai.

Ba case study dưới đây là khung ghi nhận trace, không phải kết quả chạy đã được điền. Việc để ô “chưa có trace” là chủ ý khoa học: raw Critic hoàn chỉnh và ảnh demo thật chưa đủ để chọn một câu cụ thể mà không suy diễn. Khi người dùng cung cấp log, từng bảng sẽ được điền bằng đúng ID, chunk, Điều, quyết định gate, draft và final answer tương ứng.

Bảng 13. Case study 1 — No-gap, khung trace cần bổ sung

Trường	Nội dung
Câu hỏi	Chưa chọn; phải lấy từ một record control_no_gap có raw trace đầy đủ
Mode	critic trên Base retriever
Top-k chunks/Điều ban đầu	Chưa có trace; không suy diễn từ aggregate
Evidence gap	Kỳ vọng không có; phải xác nhận bằng critic_report
Điều graph bổ sung/gate	Kỳ vọng rỗng; phải xác nhận bằng log
Draft/Final answer	Chưa có; nếu không có gap thì final bằng draft, trừ safeguard nhiều văn bản
Required facts/nhận xét	Sẽ đối chiếu theo record testset đã chọn

[CHỖ ẢNH DEMO: demo_critic_no_gap.png]

Bảng 14. Case study 2 — Same-Article gap, khung trace cần bổ sung

Trường	Nội dung
Câu hỏi	Chưa chọn; phải lấy từ record same_dieu_compound_penalty hoặc structural_multi_part
Mode	critic trên Base retriever
Top-k chunks/Điều ban đầu	Chưa có trace; cần thể hiện phần chế tài/khắc phục bị thiếu
Evidence gap	Same-Article: thiếu Khoản/Điểm, chế tài bổ sung hoặc biện pháp khắc phục
Điều graph bổ sung/gate	Parent cùng Điều; ghi chính xác yes/no/exception từ log
Draft/Final answer	Chưa có; chỉ kết luận sinh lại khi có evidence được chấp nhận
Required facts/nhận xét	Đánh dấu từng fact được draft và final bao phủ

[CHỖ ẢNH DEMO: demo_critic_same_article.png]

Bảng 15. Case study 3 — Cross-Article reference, khung trace cần bổ sung

Trường	Nội dung
Câu hỏi	Chưa chọn; phải lấy từ record cross_reference có ít nhất hai diu_ids
Mode	critic trên Base retriever
Top-k chunks/Điều ban đầu	Chưa có trace; cần nêu Điều A và phần dẫn chiếu
Evidence gap	Cross-Article qua cạnh THAM_CHIEU
Điều graph bổ sung/gate	Ghi Điều B, chiếu cạnh, hop và quyết định gate từ log
Draft/Final answer	Chưa có; phải giữ nguyên văn artefact khi bổ sung
Required facts/nhận xét	Đối chiếu fact thuộc Điều A và Điều B riêng biệt

[CHỖ ẢNH DEMO: demo_critic_cross_article.png]

4.6. Quy trình chạy đánh giá

run_evaluation.py đọc testset và lặp trên từng mode. Với mỗi record, script truyền query vào pipeline với skip_router=True, lưu answer, contexts, required facts, retrieved_dieu_ids_ranked, graph_fetched_dieu_ids, draft/critic report và hai nhóm token. Tùy chọn --resume tiếp tục batch; exception của từng câu được in rồi bỏ qua, do đó số dòng output phải được kiểm tra trước khi tổng hợp. score_evaluation.py tính retrieval metrics, Article metrics, LCR và RAGAS khi dependency/API sẵn sàng.

Bộ test hiện có 301 dòng và SHA-256 58e999c2a33f762e352ef6f477ea1ef49784b3cfa7c9b291ace24e354eba1b96. Tài liệu hướng dẫn cũ trong repo vẫn mô tả 50 câu; con số này không được dùng. Raw Base ngày 31/07/2026 hoàn chỉnh ở Naive và Expansion, nhưng Critic dừng ở 23 dòng. Vì thế, tổng hợp ba mode trong FinalFix được dùng với nhãn “source-reported”, còn MRR/nDCG tái tính từ raw Naive được báo cáo riêng để không trộn hai snapshot.

4.7. Chấm Legal Completeness

Judge nhận câu hỏi, answer và từng required fact. Với mỗi fact, judge chỉ được trả yes/no; câu trả lời tương đương về ngữ nghĩa được chấp nhận, nhưng con số và điều kiện pháp lý phải

đúng. Điểm câu là tỷ lệ fact yes; LCR tổng là trung bình phần trăm trên toàn bộ câu. Cùng một judge được dùng cho các mode để giảm biến thiên do evaluator.

Phương pháp vẫn có ba rủi ro. Thứ nhất, required facts và reference answer là silver standard. Thứ hai, judge có thể ưu tiên câu dài hoặc cách diễn đạt giống reference. Thứ ba, không có báo cáo agreement giữa judge và chuyên gia. Vì vậy, LCR được dùng để so sánh nội bộ các cấu hình dưới cùng protocol, không phải chứng nhận độ đúng pháp lý tuyệt đối.

4.8. Kiểm thử phần mềm và kiểm soát lỗi

Các kiểm tra cần thực hiện gồm: unit test cho regex chunking và ID collision; kiểm tra tổng child/parent; kiểm tra hai collection Qdrant có payload tương thích; kiểm tra graph target tồn tại hoặc được đánh dấu placeholder; kiểm tra mapping Neo4j ID sang parent Qdrant; kiểm tra mode không lấy trùng Điều; kiểm tra token counter và checkpoint resume.

Một lỗi đáng chú ý là tổng số Khoản/Điểm trong Neo4j có thể thấp hơn Qdrant nếu extraction bỏ sót cạnh cấu trúc. `find_structurally_incomplete_dieu` vì vậy nhận `total_parts_override` từ nguồn đếm độc lập và dùng giá trị lớn hơn giữa hai nguồn. Thiết kế này tránh việc graph thiếu làm Critic kết luận sai rằng retrieval đã đủ.

4.9. Khả năng tái lập

Mức tái lập hiện tại là một phần. Live repository có pipeline, testset 301 dòng, `scripts/validate_testset.py` và raw Base Naive/Expansion; phép kiểm A1-A3 có thể tái chạy. Tuy nhiên, chưa có checkpoint/output B1-B6, raw judge decision cho LCR/RAGAS, raw Critic đủ 301 câu, package lock hoàn chỉnh, thông số phần cứng, model digest, seed, latency và log API. Fine-tuned checkpoint vắng mặt; GTE index sai kích thước. Do đó, một nhóm độc lập có thể kiểm tra corpus/index và một phần retrieval, nhưng chưa thể tái tạo chính xác toàn bộ bảng LCR, RAGAS hoặc từng quyết định gate/judge.

5. KẾT QUẢ VÀ THẢO LUẬN

5.1. Độ tin cậy của bộ testset

Theo testset_validation_methodology.md, lớp A tìm thấy 0 lỗi điều_id và 0 lỗi schema category trên 301 record. Kiểm tra gần trùng gần cờ hai cặp: cat4_new1_11 với cat4_new3_03 có similarity 0,94; cat4_new2_14 với cat4_new3_04 có similarity 0,95. Tài liệu cho biết các cặp đã được rà soát thủ công và giữ lại, nhưng không kèm biên bản adjudication để kiểm tra tiêu chí giữ.

Lớp B được tài liệu mô tả là chạy bằng o3-mini trên toàn văn các Điều gốc. Tỷ lệ đạt tổng là 99,2% cho required facts bám nguồn, 99,3% cho reference answer bám nguồn, và 100% cho bao phủ facts, độ tự nhiên/rõ ràng, nhu cầu dẫn nguồn và đúng category. Ba record bị gán cờ ở B1 là cat3_04, cat1_new3_04, cat4_new2_03; trong đó cat3_04 và cat4_new2_03 cũng bị gán cờ ở B2. Gán cờ không tự động có nghĩa cả record sai: cần đọc chi tiết quyết định judge để xác định fact hoặc mệnh đề cụ thể, nhưng tập chi tiết chưa có trong workspace.

Bảng 16. Kết quả kiểm định testset do tài liệu phương pháp báo cáo

Lớp/tiêu chí	Chế tài kép	Tham chiếu chéo	Đa thành phần	Đối chứng	Toàn bộ
A1 — ID tồn tại	0 lỗi	0 lỗi	0 lỗi	0 lỗi	0 lỗi
A2 — Schema category	0 lỗi	0 lỗi	0 lỗi	0 lỗi	0 lỗi
A3 — Cặp gần trùng	—	—	—	2 cặp	2 cặp
B1 — Facts grounded	99,3%	100,0%	99,2%	99,0%	99,2%
B2 — Answer grounded	100,0%	100,0%	99,2%	99,0%	99,3%
B3 — Facts được đáp án bao phủ	100,0%	100,0%	100,0%	100,0%	100,0%
B4 — Tự nhiên, rõ ràng	100,0%	100,0%	100,0%	100,0%	100,0%
B5 — Cần dẫn nguồn	100,0%	100,0%	100,0%	100,0%	100,0%
B6 — Đúng category	100,0%	100,0%	100,0%	100,0%	100,0%

Các kết quả trên cải thiện mức minh bạch so với việc chỉ gọi testset là silver standard: đã có script, định nghĩa tiêu chí, đối chiếu toàn văn và danh sách trường hợp cần rà soát. Tuy nhiên, chúng chưa thay thế kiểm định độc lập. Không có checkpoint, raw output, model revision của lần chấm hoặc tệp summary/details/issues trong workspace; cũng chưa có chuyên gia luật đánh giá mù và đo agreement. Vì vậy, khóa luận dùng testset cho so sánh nội bộ nhưng không diễn giải các tỷ lệ 99–100% như chứng nhận “đúng pháp lý tuyệt đối”.

5.2. Kết quả truy xuất ban đầu của Base retriever

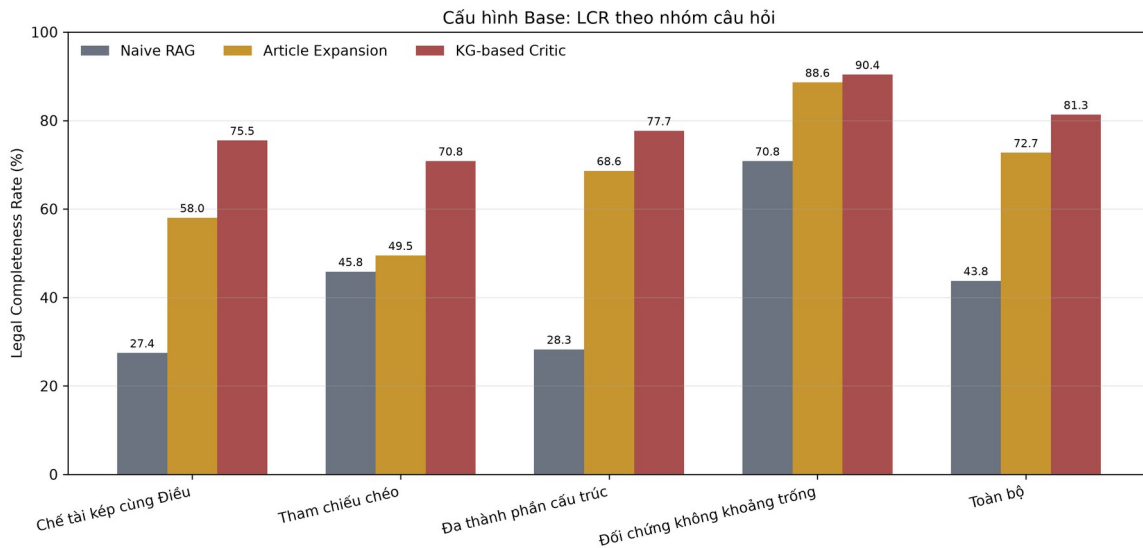
Bảng 17. Metric retrieval Base theo hai snapshot bằng chứng

Chỉ số	Tái tính từ raw Naive 301 câu	FinalFix báo cáo	Trạng thái
MRR@5	0,9028	0,8560	Hai snapshot khác nhau
nDCG@5	0,9122	0,8660	Hai snapshot khác nhau
Article precision	0,3095	0,3080	Gần nhau, không tự động coi là cùng run

Chỉ số	Tái tính từ raw Naive 301 câu	FinalFix báo cáo	Trạng thái
Article recall	0,9585	0,9520	Gần nhau, không tự động coi là cùng run

Cột thứ nhất được tái tính trực tiếp từ 301 dòng raw Base Naive có hash 0a5e...5893; cột thứ hai là aggregate trong FinalFix. Vì MRR và nDCG chênh lệch đáng kể, khóa luận không chọn âm thầm một giá trị. Kết quả chương này dùng FinalFix khi cần so sánh đồng bộ với LCR/RAGAS của ba mode, còn cột tái tính chứng minh artefact raw hiện tại thuộc snapshot khác. Không có kết luận về fine-tuning hoặc GTE trong phần kết quả chính.

5.3. Legal Completeness Rate



Nguồn: STAIS2026_Legal_RAG_FinalFix.docx; số liệu được nguồn báo cáo, chưa tái chạy đầy đủ.

Hình 8. Legal Completeness Rate do FinalFix báo cáo cho ba chế độ dùng chung Base retriever

Bảng 18. Legal Completeness Rate trên 301 câu hỏi

Nhóm câu hỏi	Số câu	Naive	Article Expansion	KG-based Critic
Chế tài kép cùng Điều	51	27,45%	58,01%	75,49%
Tham chiếu chéo	16	45,83%	49,48%	70,83%
Đa thành phần cấu trúc	130	28,27%	68,64%	77,67%
Đối chứng không khoảng trống	104	70,83%	88,62%	90,38%
Toàn bộ	301	43,77%	72,72%	81,33%

Theo FinalFix, Article Expansion cao hơn Naive 28,95 điểm phần trăm trên toàn bộ testset; Critic cao hơn Expansion 8,61 điểm và cao hơn Naive 37,56 điểm. Nhóm tham chiếu chéo tăng từ 49,48% ở Expansion lên 70,83% ở Critic, là chênh lệch 21,35 điểm và phù hợp với mục đích của THAM_CHIEU. Nhóm chế tài kép và đa thành phần cũng cải thiện khi khôi phục parent Article. Tuy nhiên, đây là liên hệ mô tả giữa kiến trúc và metric, không phải bằng chứng nhân quả, vì chưa có ablation, repeated runs hoặc paired test. Nhóm no-gap chỉ tăng 1,76 điểm từ Expansion lên Critic; thiếu raw Critic đầy đủ nên chưa xác định được bao nhiêu trường hợp gate/regeneration làm tốt hơn hoặc làm xấu đi một câu vốn đã đủ context.

5.4. Kết quả RAGAS

Bảng 19. Kết quả RAGAS Base do FinalFix báo cáo

Chỉ số	Naive	Article Expansion	KG-based Critic
Faithfulness	0,705	0,841	0,857
Answer Relevancy	0,349	0,597	0,593
Context Precision	0,965	0,963	0,961
Answer Correctness	0,544	0,657	0,668

Faithfulness và Answer Correctness tăng theo hai mode bổ sung cấu trúc; Answer Relevancy của Expansion và Critic gần nhau, còn Context Precision giảm nhẹ từ 0,965 xuống 0,961. Kết quả minh họa rằng độ bao phủ facts, tính trung thực, độ liên quan và độ tập trung của context là các chiều khác nhau. Toàn bộ bảng là số liệu FinalFix báo cáo; workspace chưa có raw RAGAS output và evaluator trace để tái tính, nên chênh lệch nhỏ không được diễn giải như có ý nghĩa thống kê.

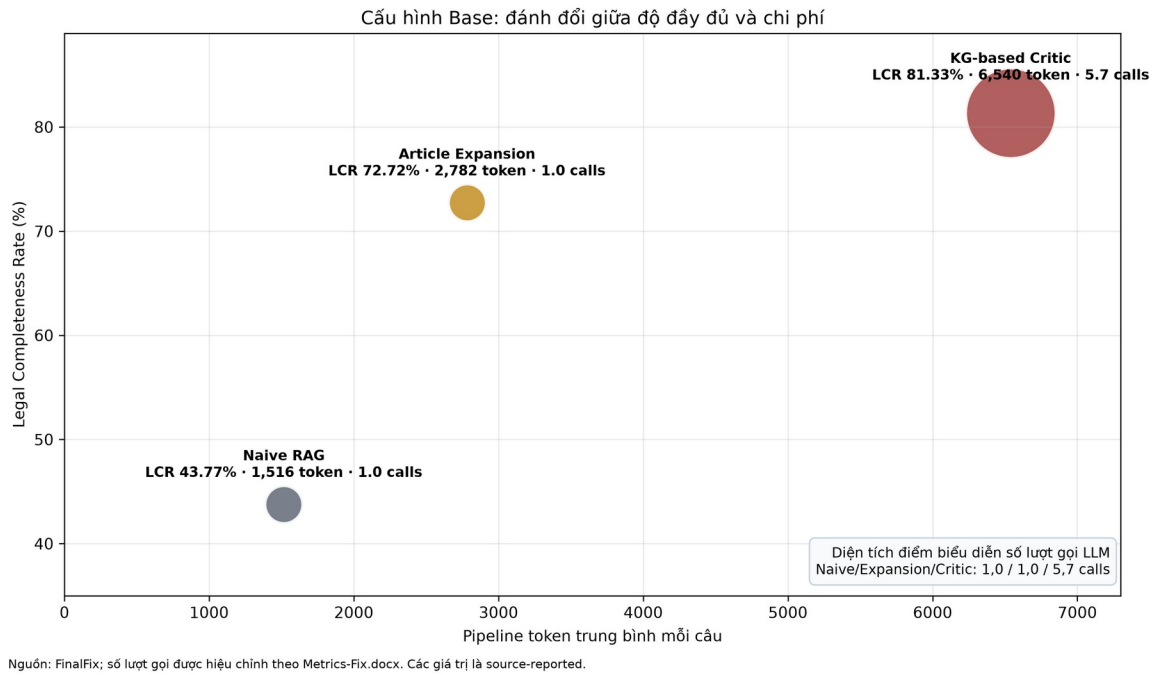
5.5. Precision, recall và độ dài ngữ cảnh theo Điều

Bảng 20. Chỉ số context ở cấp Điều

Chỉ số	Naive	Article Expansion	KG-based Critic
Article precision	0,308	0,303	0,308
Article recall	0,952	0,952	0,962
Context characters	1.594	8.296	4.748

Theo FinalFix, Critic có Article recall cao nhất 0,962 trong khi precision bằng Naive và cao hơn nhẹ Expansion. Diễn giải thận trọng là context cuối của Critic chứa thêm Điều đúng mà aggregate không cho thấy giảm precision; chưa thể khẳng định ý nghĩa thống kê. Article Expansion tạo context dài nhất 8.296 ký tự. Critic dùng 4.748 ký tự, thấp hơn 42,8% so với Expansion nhưng vẫn cao hơn Naive. Đây là điểm cân bằng mô tả giữa độ phủ và lượng nội dung, không phải tối ưu đã được chứng minh.

5.6. Chi phí token và số lượt gọi mô hình



Hình 9. Đánh đổi LCR, pipeline token và số lượt gọi mô hình của cấu hình Base

Bảng 21. Chi số tài nguyên trung bình trên mỗi câu hỏi

Chỉ số	Naive	Article Expansion	KG-based Critic
Pipeline tokens/câu	1.516	2.782	6.540
Final-call tokens/câu	1.352	2.618	2.187
LLM calls/câu	1,0	1,0	5,7

Critic tiêu thụ nhiều nhất trên toàn pipeline: 6.540 token và 5,7 lượt gọi trung bình, so với 2.782 token và 1,0 lượt của Expansion. Các số calls 2,0/2,0/6,6 ở một bản FinalFix cũ mâu thuẫn với cách chạy evaluation bỏ router; khóa luận dùng 1,0/1,0/5,7 từ Metrics-Fix và ghi rõ nguồn hiệu chỉnh. Final-call token của Critic là 2.187, thấp hơn 2.618 của Expansion, nhưng điều này không làm Critic rẻ hơn vì còn draft và gate. Hình và bảng đều dùng số liệu source-reported, không suy ra latency hoặc chi phí tiền tệ.

5.7. Trả lời các câu hỏi nghiên cứu

5.7.1. RQ1 - Khác biệt giữa ba mode

Trên Base retriever, Naive có LCR 43,77%; Article Expansion đạt 72,72%; Critic đạt 81,33%. Nhóm tham chiếu chéo tăng 21,35 điểm từ Expansion lên Critic. Các kết quả phù hợp với giả thuyết rằng top-5 child thiếu cấu trúc và graph có thể gợi ý Điều được dẫn chiếu. Vì số liệu là aggregate do nguồn báo cáo và không có ablation, kết luận dừng ở mức liên hệ mô tả, không khẳng định quan hệ nhân quả.

5.7.2. RQ2 - Ảnh hưởng của khôi phục và sửa bằng chứng

Article Expansion giữ Article recall ở 0,952 nhưng tăng context từ 1.594 lên 8.296 ký tự; Critic nâng recall lên 0,962 với context 4.748 ký tự. Faithfulness tăng từ 0,705 lên 0,841 và

0,857; Answer Correctness tăng từ 0,544 lên 0,657 và 0,668. Context Precision giảm nhẹ. Kết quả cho thấy bổ sung cấu trúc có liên hệ với độ phủ và độ trung thực cao hơn, nhưng chưa chứng minh từng thành phần graph hoặc gate là nguyên nhân vì thiếu ablation.

5.7.3. RQ3 - Đánh đổi chất lượng và chi phí

Critic đổi thêm compute trung gian để lấy LCR và Article recall cao hơn. Pipeline token tăng từ 2.782 ở Expansion lên 6.540 và số lượt gọi tăng từ 1,0 lên 5,7; final-call token lại giảm từ 2.618 xuống 2.187. Vì vậy, Naive phù hợp luồng tối giản, Expansion phù hợp bài toán chủ yếu thiếu phần trong cùng Điều với một lần sinh, còn Critic phù hợp hơn khi ưu tiên tính đầy đủ hoặc cross-reference và chấp nhận chi phí cao hơn.

5.8. Phân tích lỗi và rủi ro

Retriever có thể đưa Điều nhiều vào top-k. Nếu Critic kiểm tra mọi Điều, mỗi Điều nhiều có thể kéo theo tham chiếu riêng và làm context phình to. Ngưỡng tương đối và giới hạn bốn Điều là cơ chế giảm rủi ro này. Tuy nhiên, ngưỡng cố định có thể bỏ sót câu hỏi thực sự cần nhiều Điều điểm thấp hơn; cần ablation để chọn giá trị.

Graph extraction có thể bỏ entity, sai relation hoặc tạo placeholder không thể resolve. Đặc biệt, một member extraction bị trùng/misabeled cho thấy kiểm soát artefact chưa đủ. Critic có override số phần từ Qdrant, nhưng THAM_CHIEU vẫn phụ thuộc graph. Một graph sai có thể bỏ sót Điều cần lấy hoặc lấy Điều không liên quan.

Semantic gate có thể false negative và loại bằng chứng cần thiết, hoặc false positive và chấp nhận context nhiều. Cấu hình fail-open bảo vệ recall nhưng làm tăng rủi ro nhiều khi model/API lỗi. Production nên phân biệt timeout, parse error và quyết định yes; exception hiện hành không được xem như bằng chứng liên quan đã được model xác nhận.

LLM judge có thể thiên lệch theo độ dài. Critic thường tạo answer/context khác độ dài so với các mode khác, nên bias độ dài có thể ảnh hưởng LCR. Cần đánh giá mù mode, đảo thứ tự, lấy mẫu chuyên gia và đo agreement để kiểm tra.

5.9. Giới hạn nghiên cứu

Thứ nhất, kết quả LCR, RAGAS và chi phí hiện có là aggregate trên 301 câu do nguồn báo cáo, không có repeated runs, confidence interval hoặc paired statistical test. Raw Critic hiện chỉ có 23 record. Mọi so sánh là mô tả, không phải bằng chứng ý nghĩa thống kê.

Thứ hai, live repo có script kiểm định testset và phép kiểm A1-A3 đã được tái chạy, nhưng không có checkpoint, output B1-B6 hoặc biên bản rà soát hai cặp gần trùng. Quy trình xây dựng câu hỏi, vai trò người chú giải, trình độ pháp lý, guideline, inter-annotator agreement và adjudication cũng chưa được tài liệu hóa đầy đủ. Required facts và reference answer vì vậy vẫn là silver standard có kết quả tự động được nguồn báo cáo, chưa phải gold standard chuyên gia.

Thứ ba, LCR chưa được validation với luật sư; raw fact decision và error log của judge chưa được cung cấp đầy đủ. RAGAS phụ thuộc phiên bản và evaluator, nên cần lưu prompt và output từng câu.

Thứ tư, corpus chỉ bao phủ một số lĩnh vực và năm PDF chưa vào index. Phép chạy chunker và index hiện hành xác nhận 1.211 parent; manifest 1.209 là tài liệu cũ mâu thuẫn. Văn bản còn thay đổi hiệu lực, do đó kết quả phản ánh snapshot thí nghiệm, không mặc định là luật hiện hành tại mọi thời điểm.

Thứ năm, chưa có đánh giá độc lập chất lượng KG. Artefact extraction có một file trùng/mislabeled; số node/cạnh và precision/recall relation không được công bố. Không thể tách đóng góp của same-Article check, graph traversal, semantic gate và regeneration vì chưa có component ablation.

Thứ sáu, chưa có môi trường thực nghiệm khóa phiên bản hoàn chỉnh: hardware, OS/CUDA, package lock, model digest, seed, latency, energy và monetary cost. Fine-tuned checkpoint không tồn tại; GTE index hiện tại sai kích thước so với model card. Các metric token cũng thiếu định nghĩa chi tiết về cached/system/tool token.

5.10. Hàm ý thiết kế

Kết quả gợi ý không nên coi “nhiều context hơn” là chiến lược mặc định. Parent expansion giải quyết tốt thiếu cấu trúc cùng Điều nhưng tăng mạnh độ dài. Graph critique có thể dùng như lớp kiểm tra sau retrieval, đặc biệt cho domain có quan hệ dẫn chiếu rõ. Tuy nhiên, để đáng tin cậy, graph cần provenance, version và đánh giá riêng; gate cần log và chính sách lỗi rõ ràng.

Một kiến trúc production có thể dùng routing theo rủi ro. Câu hỏi đơn giản chạy naive; câu hỏi có nhiều điều kiện cùng Điều chạy expansion; câu hỏi có dấu hiệu dẫn chiếu hoặc chế tài chạy critic. Routing này phải được đánh giá trên dữ liệu giữ lại, không thiết kế từ chính testset 301 câu.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Khóa luận đã trình bày một hệ thống hỏi đáp pháp luật tiếng Việt kết hợp chunking Điều - Khoản - Điểm, hybrid retrieval, parent Article reconstruction, Knowledge Graph và Critic Agent. Điểm cốt lõi là thay đổi vai trò của graph: graph không trực tiếp thay văn bản để trả lời, mà phát hiện vị trí có khả năng thiếu và hướng dẫn hệ thống lấy thêm parent Article. Semantic gate tiếp tục lọc ứng viên, còn regeneration chỉ xảy ra khi có bằng chứng được chấp nhận.

Trên số liệu tổng hợp của 301 câu hỏi dùng Base retriever, FinalFix báo cáo LCR 43,77% cho Naive, 72,72% cho Article Expansion và 81,33% cho Critic. Nhóm tham chiếu chéo cải thiện rõ nhất giữa Expansion và Critic, phù hợp với mục tiêu duyệt THAM_CHIEU. Critic dùng 6.540 pipeline token và 5,7 model call trung bình, cao hơn hai baseline, nhưng final-call token thấp hơn Expansion. Tài liệu kiểm định testset báo cáo A1-A2 không có lỗi, B1-B2 đạt 99,2-99,3% và B3-B6 đạt 100%; khóa luận giữ riêng giới hạn rằng chưa có output B1-B6 hoặc xác nhận chuyên gia để tái kiểm. Raw Critic chỉ có 23/301 dòng, nên kết luận từ LCR/RAGAS được gắn nhãn source-reported. Fine-tuned và GTE bị loại khỏi trực chính vì thiếu checkpoint hoặc index không hợp lệ.

Các kết luận được giới hạn ở phạm vi snapshot, protocol và metric đã có. Khóa luận không khẳng định hệ thống thay thế chuyên gia luật, không suy luận ý nghĩa thống kê từ aggregate, và không xem LLM judge là thẩm định pháp lý.

6.2. Hướng phát triển

Hướng ưu tiên thứ nhất là hoàn thiện reproducibility package: chạy đủ 301 câu cho ba mode Base, phát hành ranked lists, context, draft, critic trace, gate decision, prompt, judge decision và log token; lưu checkpoint cùng summary/details/issues của kiểm định testset; khóa phiên bản dependency, model digest/revision, corpus và từng index bằng hash.

Hướng thứ hai là xây dựng gold standard có chuyên gia. Cần guideline, ít nhất hai người chú giải độc lập, measurement agreement và adjudication. Một mẫu nên được double-blind so sánh với GPT-4o-mini judge để ước lượng false positive/negative của LCR.

Hướng thứ ba là sửa và đánh giá KG. Archive cần thay file trùng/mislabeled, resolve placeholder, xuất graph immutable và đo precision/recall trên mẫu node/relation. Ablation riêng cho compound-penalty check, structural check, THAM_CHIEU, gate và regeneration sẽ xác định đóng góp của từng thành phần.

Hướng thứ tư là quản lý thời gian hiệu lực. Mỗi chunk nên có effective_from, effective_to, supersedes, nguồn chính thức và ngày tái. Retrieval cần lọc theo thời điểm người dùng hỏi. Khi văn bản được sửa đổi, pipeline phải cập nhật Qdrant và Neo4j trong cùng transaction/version.

Hướng thứ năm là tối ưu chi phí. Có thể dùng model gate nhỏ hơn, batch candidate, cache graph query, hoặc router rủi ro để chỉ gọi Critic khi câu hỏi có khả năng thiếu cấu trúc. Cần báo cáo latency và monetary cost song song với token.

Hướng cuối là phát triển API và giao diện có khả năng giải thích. Câu trả lời nên hiển thị Điều/Khoản/Điểm, văn bản, ngày hiệu lực, context đã dùng và cảnh báo khi graph/gate không chắc chắn. Người dùng phải có đường dẫn tới văn bản gốc và cơ chế phản hồi để phát hiện lỗi.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [2] P. M. Kien, H.-T. Nguyen, N. X. Bach, V. Tran, M. L. Nguyen, and T. M. Phuong, “Answering legal questions by learning neural attentive text representation,” in *Proc. 28th Int. Conf. Computational Linguistics*, pp. 988-998, 2020, doi: 10.18653/v1/2020.coling-main.86.
- [3] S. Khazaeli, J. Punuru, C. Morris, S. Sharma, B. Staub, M. Cole, S. Chiu-Webster, and D. Sakalley, “A free format legal question answering system,” in *Proc. Natural Legal Language Processing Workshop*, pp. 107-113, 2021, doi: 10.18653/v1/2021.nllp-1.11.
- [4] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157-173, 2024, doi: 10.1162/tacl_a_00638.
- [5] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract-Conference.html
- [6] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008, doi: 10.1017/CBO9780511809071.
- [7] X. Zhang *et al.*, “mGTE: Generalized long-context text representation and reranking models for multilingual text retrieval,” in *Proc. EMNLP 2024: Industry Track*, pp. 1393-1412, 2024, doi: 10.18653/v1/2024.emnlp-industry.103.
- [8] N. Guha *et al.*, “LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023.
- [9] Z. Fei, X. Shen, D. Zhu, F. Zhou, Z. Han, A. Huang, S. Zhang, K. Chen, Z. Yin, Z. Shen, J. Ge, and V. Ng, “LawBench: Benchmarking legal knowledge of large language models,” in *Proc. EMNLP*, pp. 7933-7962, 2024, doi: 10.18653/v1/2024.emnlp-main.452.
- [10] J. Lee, D. Kim, S. Hwang, H. Kim, and G. Lee, “KoBLEX: Open legal question answering with multi-hop reasoning,” in *Proc. EMNLP*, pp. 4019-4053, 2025, doi: 10.18653/v1/2025.emnlp-main.200.
- [11] A. B. Hou, O. Weller, G. Qin, E. Yang, D. Lawrie, N. Holzenberger, A. Blair-Stanek, and B. Van Durme, “CLERC: A dataset for U.S. legal case retrieval and retrieval-augmented analysis generation,” in *Findings of NAACL*, pp. 7913-7928, 2025, doi: 10.18653/v1/2025.findings-naacl.441.
- [12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150-158, 2024, doi: 10.18653/v1/2024.eacl-demo.16.
- [13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proc. NAACL*, pp. 338-354, 2024, doi: 10.18653/v1/2024.naacl-long.20.
- [14] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, Datasets and Benchmarks Track, 2023.
- [15] Alibaba-NLP, “gte-multilingual-base,” *Hugging Face*, 2024. [Online]. Available: <https://huggingface.co/Alibaba-NLP/gte-multilingual-base>. [Accessed: Jul. 21, 2026].

PHỤ LỤC A. BẢNG ĐỐI CHIẾU BẰNG CHỨNG

Bảng 22. Nguồn nội bộ dùng cho các khẳng định thực nghiệm

Nội dung	Artefact có thẩm quyền	Ghi chú
Số liệu Base và hiệu chỉnh lượt gọi	STAIS2026_Legal_RAG_FinalFix.docx, Metrics-Fix.docx	LCR/RAGAS/resource là source-reported; calls dùng 1/1/5,7
Cấu hình GTE lịch sử	gte_base.md	Dùng xác định chiều đúng 768 và phát hiện index hiện tại không hợp lệ
Mã nguồn hiện hành	/Users/nguyengiahuy/Project/LEGAL_IT_CHATBOT	Nguồn ưu tiên cho tham số, nhánh và giao diện
Chỉ mục Base	data/.qdrant_base, data/bm25_base.json	1.211 parent, 10.355 child, dense 1.024
Testset	data/eval_testset.jsonl	301 dòng, hash đã ghi ở Chương 4
Phương pháp kiểm định testset	testset_validation_methodology.md	Nguồn cho 9 tiêu chí, tỷ lệ tổng hợp và ID bị gán cờ; chưa có output thô đi kèm
Source snapshot cũ	tmp/final-round/origin-main/src/	Chỉ dùng phát hiện mâu thuẫn, không ghi đè live code
Nguồn khoa học	Trang xuất bản NeurIPS/ACL/Cambridge và model card	Đối chiếu tiêu đề, tác giả, năm, trang và DOI/URL

PHỤ LỤC B. DANH SÁCH GIỚI HẠN TÁI LẬP CẦN BỔ SUNG

1. Manifest corpus theo file, URL nguồn, ngày tải, trạng thái hiệu lực và hash.
2. Bản sửa artefact extraction bị trùng/mislabeled và graph export bất biến.
3. Training dataset, split, hyperparameter, seed và checkpoint hash của fine-tuned retriever.
4. Ranked list, context, answer, draft, critic report và gate decision theo từng câu.
5. Prompt, raw decision, retry/failure policy và validation chuyên gia cho LCR judge.
6. Prompt và raw output của RAGAS 0.1.22 cho từng cấu hình.
7. Quy tắc macro/micro, duplicate và zero denominator của Article precision/recall.
8. Tokenizer và ranh giới tính token cho pipeline/final call.
9. Hardware, OS/CUDA, package lock, latency, energy và chi phí tiền tệ.
10. Immutable revision, max_seq_length, collection name/hash và ingest log của GTE.
11. Repeated runs, confidence interval, paired test và component ablation.
12. Checkpoint, summary/details/issues, raw judge output, model revision và biên bản rà soát thủ công của kiểm định testset.

PHỤ LỤC C. TUYÊN BỐ VỀ VIỆC SỬ DỤNG CÔNG CỤ AI

Bản khóa luận được hoàn thiện với sự hỗ trợ của công cụ AI trong việc tổ chức cấu trúc, biên tập ngôn ngữ, đối chiếu nội dung và định dạng tài liệu. Các số liệu thực nghiệm chỉ được lấy từ artefact trong workspace; tài liệu tham khảo được kiểm tra trên nguồn xuất bản chính thức. Tác giả có trách nhiệm đọc lại toàn bộ nội dung, xác nhận tính chính xác của mô tả triển khai, kiểm tra quy định đào tạo của cơ sở và chịu trách nhiệm cuối cùng đối với bản nộp.